

- **Artefactos y Componentes:** En esta vista, identificamos los **artefactos** (como componentes y módulos de implementación) que se utilizan para ensamblar el sistema. Estos artefactos incluyen código fuente, bibliotecas y otros elementos necesarios para la implementación.
- **Correspondencia:** La vista también establece la correspondencia entre los **elementos de software** (como clases y componentes lógicos) y los artefactos físicos. Es decir, cómo se traducen los componentes lógicos en implementaciones concretas.
- **Propósito:** La Vista de Implementación toma decisiones sobre cómo estructurar el repositorio donde se almacenarán los componentes de código. También aborda el control de cambios, la configuración y las políticas de gestión de configuración. En resumen, es fundamental para la gestión efectiva del desarrollo y la implementación del software.

La Vista de Despliegue.

Se enfoca en la distribución física del sistema y cómo se ejecuta en la infraestructura de hardware (HW). Es fundamental para comprender cómo se implementa el sistema en la infraestructura física y cómo se distribuyen los componentes de software en los nodos de hardware.

- **Topología de Hardware:** Describe cómo los nodos de hardware (como computadoras y dispositivos) se organizan para formar la infraestructura sobre la cual se ejecuta el sistema. Esto incluye la disposición de servidores, estaciones de trabajo, dispositivos móviles, etc.
 - **Niveles de Hardware:** La vista muestra los diferentes niveles de hardware involucrados. Por ejemplo, puede haber servidores de aplicación, servidores de base de datos, dispositivos de red, etc. Cada nivel tiene su función específica en el sistema.
 - **Componentes de Software:** También se considera cómo los componentes de software (como módulos, servicios o aplicaciones) se alojan en los nodos de hardware. Por ejemplo, qué parte del sistema se ejecuta en qué servidor o dispositivo.
 - **Distribución y Comunicación:** La Vista de Despliegue se ocupa de la distribución, entrega e instalación de las partes físicas del sistema. Además, muestra cómo se comunican los diferentes niveles de hardware entre sí y con los componentes de software.
-

PARCIAL 3 – PATRONES DE DISEÑO.

Conceptos básicos:

- **Interdependencia Funcional:** la funcionalidad de un componente debe ser cohesiva y centrarse en una y sólo una función. El **concepto de interdependencia funcional es resultado de la separación de problemas y de los conceptos de abstracción y ocultamiento de información.** Se logra desarrollando **módulos** donde cada uno **resuelva un subconjunto específico de requerimientos** y tenga una interfaz sencilla. *El SW con modularidad eficaz, es decir, con módulos independientes, es más fácil de desarrollar porque su función se subdivide y las interfaces se simplifican.* Los **módulos independientes son más fáciles de mantener** (y probar) debido a que los efectos secundarios causados por el diseño o por la modificación del código son limitados, se reduce la propagación del error y es posible obtener módulos **reutilizables**. La independencia se evalúa con el uso de dos criterios:

Cohesión: un **módulo cohesivo ejecuta una sola tarea**, por lo que requiere interactuar poco con otros componentes en otras partes del programa.

Acoplamiento: es un **indicador de la independencia relativa entre módulos**. En el diseño de SW, debe buscarse el **mínimo acoplamiento posible**.

- **Reusabilidad:** grado en el que un programa puede volverse a utilizar en otras aplicaciones (se relaciona con el empaque y el alcance de las funciones que lleva a cabo el programa).
- **Modularidad:** construir SW que tenga modularidad eficaz, **cada módulo debe centrarse exclusivamente en un aspecto bien delimitado del sistema**: debe ser cohesivo en su función o restringido en el contenido que representa. Los módulos deben estar interconectados sencillamente: poco acoplamiento. La modularidad es la manifestación más común de la división de problemas. El SW se divide en componentes con nombres distintos y abordables por separado, llamados módulos, que se integran para satisfacer los requerimientos del problema.

Delegación: significa pedirle a alguien que haga algo por mí, confiar y darle el 100% de la responsabilidad para que lo realice y me devuelva el resultado. **Mecanismo que permite polimórficamente aprovechar un código que está escrito en otra clase y obtener el resultado sin tener que romper el encapsulamiento de esta clase (Caja negra)**, evitando escribir el mismo código, y solo realizando un llamado o mensaje. El contexto de relación que permite la delegación es la asociación, agregación y composición. **Delegar la responsabilidad a un objeto es diferente a pedirle colaboración. Lo usan patrones como State, Strategy, composite, etc.**

Permite hacer reutilización con composición en lugar de herencia. Un objeto le delega una tarea a otro, resolviendo una petición entre 2 objetos o más. La idea es componer un objeto con instancias de otros, y permitirle que les pida a éstos que resuelvan cosas por él, logrando tener un “pseudo objeto” más complejo. La relación es sumamente flexible ya que los delegados pueden cambiar en tiempo de ejecución. Si se utilizan interfaces en lugar de implementaciones, el objeto contenedor no tiene por qué saber cómo se van a resolver sus peticiones, las delega y otro le entregará los resultados.

Herencia – mecanismo de reutilización a nivel de clases, heredando atributos/comportamientos de otra, jerarquía de clases.

Composición – mecanismo de reutilización a nivel de objetos, construcción de objetos complejos a partir de objetos más simples. La composición es preferida sobre la herencia cuando se busca flexibilidad y mantenibilidad, ya que permite cambiar el comportamiento de un objeto en tiempo de ejecución.



PRINCIPIOS DE DISEÑO DE SOFTWARE ORIENTADO A OBJETOS.

Principio de diseño es una técnica o herramienta básica que se aplica para construir SW y que nos permiten evitar tener un mal diseño, obteniendo SW más flexible, extensible y mantenible. Son guías de alto nivel, que aplican a cualquier lenguaje de programación OO. *Su foco principal es mantener la cohesión alta, el acoplamiento bajo, facilitar los cambios y prevenir errores inesperados.*

- ❖ **DRY (Don't Repeat Yourself - No te repitas):** busca que cada funcionalidad del sistema esté representada una sola vez en el código. Así el SW se puede modificar sin romper partes que no se relacionen a lo que se cambia, y se mantiene consistente al no correr el riesgo de que la misma funcionalidad se haga de dos maneras diferentes.

Patrón que aplica: State, porque todo el código a implementar por los estados concretos está definido en la clase abstracta, entonces cada una solo redefine agregando funcionalidad específica, sin duplicar.

El objetivo del principio es evitar la duplicación del código, definiéndolo en un único lugar, de forma tal que creamos abstracciones. Permite que si algo cambia en el SW solo se modifique en donde corresponda, sin repercutir en todo el SW, el código se mantiene mucho más limpio.

- ❖ **SOC (Separation of concerns) →** Separación de intereses, aislar la aplicación de software en secciones separadas. Cada sección debe abordar una preocupación distinta que tenga mínima superposición con las demás.

- Gestiona la complejidad y aumenta la mantenibilidad.
 - Mejora la cohesión y reduce el acoplamiento
 - Facilita la extensión OCP
 - Permite la reutilización
- Reduce la fragilidad de los módulos

- ❖ **YAGNI (You aint gonna need it) →** No lo vas a necesitar, recalcando la pérdida de tiempo que trae crear código para usarlo en un futuro.

- ❖ **KISS (Keep it simple, stupid) →** Mayor calidad de código simple y fácil de mantener y mejorar. Implicaciones de este principio.

- No aumentar infinitamente el nivel de abstracción → incrementa mucho la complejidad.
- No incluir funciones que no se usen (YAGNI)
- Qué sentido tiene incluir librerías de gran tamaño si al fin y al cabo se usa solo una función de ella.
- Se recomienda descomponer la complejidad en partes más simples, sosteniéndose sobre el principio DRY.

- ❖ **Principle of least knowledge - Ppio de menor conocimiento (LEY DE DEMETER) →** producir objetos que estén débilmente acoplados.

Los objetos fuertemente acoplados dependen en gran medida unos de otros y requieren una gran cantidad de información entre sí para que los métodos funcionen.

Los objetos débilmente acoplados tienen un número mínimo de dependencias y saben poco sobre los métodos de otras clases (**DONT TALK TO STRANGERS**).

- ❖ **Tell, Dont ask (Tell Don't Ask - Ordena, no preguntes) →** El POO se trata de objetos que colaboran entre sí para implementar un comportamiento que solos no podrían, y el principio

fundamental de esa colaboración es la distribución de responsabilidades, entonces un objeto le pide a otro objeto que haga algo que necesita y le devuelve el resultado. Este principio habla de distribución de responsabilidades: no tener clases que solo sean get y set, en lugar de ello, decirle al objeto qué hacer.

Si un objeto tiene los datos necesarios para realizar una función, que sea el responsable.

Esto **mejora la cohesión** de cada clase **al encapsular el código con los datos que utiliza**, hace que los objetos trabajen de una forma más equilibrada. Se relaciona con el Patrón Experto. No usar objetos para pedirles cosas, sino decirles que las hagan.

Patrón que aplica: State (cuando se delegan las responsabilidades al estado actual del objeto), **Iterator** (cuando se le delega al objeto IteratorConcreto la responsabilidad y tarea de recorrer y acceder a c/ elemento de otro objeto agregado).

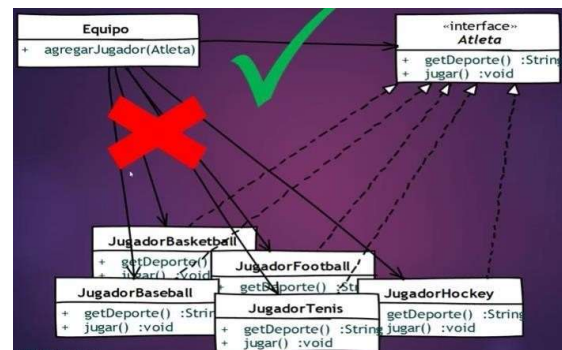
❖ **Program to interface principle** → refiere a la práctica de diseñar sistemas desacoplados y flexibles.

Literalmente es programar hacia la interfaz y no hacia la implementación, depender de abstracciones, no de clases concretas. Busca reducir el acoplamiento entre módulos o sistemas (contexto no defina tantos métodos como clases existan, sino que tenga uno solo apuntando a la interfaz), **relacionarse con abstracciones y no con concreciones.** Se define primero la interfaz o contrato que el módulo deberá proveer, y luego se implementa la funcionalidad necesaria para realizar ese contrato. Así no se expone la funcionalidad interna, el módulo cliente tiene un contrato claro de qué es lo que puede hacer.

- Usar interfaces para desacoplar el sistema propio de un sistema externo (boundary)
- Para múltiples implementaciones concretas se utiliza una interfaz que muestre todo.
- Facilita la extensión y modificación del código sin afectar otras partes del sistema.

Patrón Strategy: aquí se aplica al establecer una interfaz única, donde todas las Estrategias deben ser implementadas hacia ella para poder ser utilizadas en el Contexto. Nos interesa saber **QUE hace, no COMO** lo hace. La interfaz está definida por su signatura (nombre, parámetro, retorno y tipo), los métodos de una interfaz son vacíos. La interfaz define un contrato, el contrato que define son los métodos, o sea cualquiera que realice la interfaz tiene que poder responder a esos métodos.

- El contrato se establece entre la interfaz y las clases concretas que la implementan (las que definen el CÓMO se hace, el código).



Las interfaces me aseguran que un objeto va a poder realizar una acción, entonces cualquier cliente que necesite de esa acción va a poder pedirselo a través de esa interfaz. **Cada clase que implemente una interfaz debe cumplir con los comportamientos de esta.**

❖ **COI (Composition Over Inheritance - Composición por sobre herencia):** favorecer la composición sobre la herencia. **La herencia**, si bien es la forma **más simple** de reutilizar el código, **presenta** algunos **problemas**:

- **Genera fuerte acoplamiento** entre las clases hijas y el padre, y rompe el encapsulamiento, ya que ellas acceden a los atributos del padre.

- Al sobre escribir métodos heredados, se tiene que asegurar que el nuevo **comportamiento** sea **compatible** con el básico.
- **Subclases acopladas** estrechamente a las superclases.
- La reutilización por herencia puede conducir a crear **jerarquías paralelas** de herencia.
- Las **subclases no pueden “reducir”** (quitar métodos o no usarlos) la interfaz de la superclase.

COI→ Esto puede resolverse mejor a través de **composición**, que nos permite reutilizar comportamiento a través de la **delegación**: manteniendo un acoplamiento menor, y además se aplica el concepto de “caja negra” entre los objetos, manteniendo el encapsulamiento. Usar la delegación como reuso en vez de utilizar la herencia.

A través de la delegación usamos polimorfismo, las clases están encapsuladas y pueden tener el mismo método (signatura), y los objetos se van a comportar de acuerdo a su clase y los parámetros que se le envíen. En lugar de que un objeto implemente un comportamiento en sí mismo, se **delega** hacia otros. **Patrón que aplica: Strategy**

Beneficios: se resuelve el comportamiento de los objetos en tiempo de ejecución, dinámicamente, dependiendo solamente de las relaciones entre objetos.

	HERENCIA	REALIZACIÓN (Interfaces)	COMPOSICIÓN
Propósito	Extender una clase base con clases hijas que reutilicen su comportamiento y agreguen el propio.	Definir una interfaz (contrato) común, para que otras clases puedan cumplirlo y así unirse bajo un contrato común. Define qué hacer y no cómo.	Contener clases dentro de otras clases, para que las contenedoras puedan usar los métodos expuestos por las contenidas.
Características	- Relación fuerte de generalización en donde un objeto especificado se basa en la especificación de otro (comparten código). Una clase padre comparte sus métodos y atributos con sus clases hijas. - Esta relación permite definir una implementación en términos de otra, lo que se conoce como reutilización de caja blanca, ya que la estructura interna de las clases es visible (viola encapsulamiento). - No requiere interfaz ya que puede acceder a los miembros del padre.	- Relación débil entre clasificadores en donde un objeto clasificador establece un contrato que garantiza que otro lo cumplirá. Se utiliza entre interfaces y colaboraciones. - Se utiliza para representar la relación entre una interfaz y la clase que la implementa. - Describe cuándo se puede usar un objeto en lugar de otro. - Clases donde los métodos no tienen comportamiento, consiste solo en la firma (nombre, atributos, retorno, tipo) - No puede tener	- Relación todo-parte, a diferencia de la agregación, las partes no tienen vida independiente fuera del todo, cada parte pertenece a al menos un y sólo un todo. En agregación una parte se puede compartir entre los todos. - La funcionalidad se obtiene componiendo objetos, requiriendo que estos tengan interfaces bien definidas. Esto se conoce como reutilización de caja negra, porque los detalles internos de los objetos no son visibles. - Requiere que la clase tenga una interfaz bien definida para poder usarla.

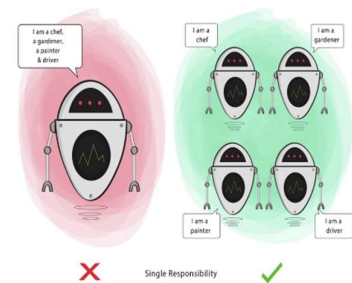
		comportamiento, todos sus métodos son abstractos.	
Polimorfismo	Si	Si	No directamente por sí sola.
Tipo de Relación	es un	es un	tiene un
Se puede modificar en	Compilación (cuando programo). Trabaja con clases	Compilación	Ejecución (el sistema se ejecuta). Trabaja con Objetos
Permite reuso de código	Si	No	Si
Permite encapsulam.	No	N/A	Si
Acoplamiento	Alto	Bajo	Medio/Bajo
Multiplicidad	1..1 En general solo se puede heredar una clase	1..*	1..* Se puede agregar muchas clases, obteniendo así múltiple reutilización
Ventajas	Fácil de implementar, porque se tiene acceso al contenido de la clase padre. Permite cambiar las <i>implementaciones</i> heredadas de las clases Aumenta la reutilización de código	Mucho más flexible y robusta que la herencia Proporciona un mecanismo para establecer contratos	Se define en tiempo de <i>ejecución</i> (<i>flexible</i>) No rompe el encapsulamiento Posee bajo acoplamiento Más consistente
Desventajas	Rompe el encapsulamiento Produce alto acoplamiento No permite modificar las implementaciones en tiempo de ejecución, ya que se define en tiempo de compilación (<i>rígida</i>)	No proporciona reutilización de código	No es polimórfica por sí misma Construir un sistema basado en composición es mucho más costoso y difícil de implementar Más difícil de implementar porque trabaja únicamente con lo que la clase expone, a través de la delegación.

PRINCIPIOS SOLID.

Son un conjunto de principios y buenas prácticas usadas en el diseño y POO, que facilitan escribir SW de calidad, legible, escalable y fácilmente testeable.

Single Responsibility Principle (SRP): Cada clase debe tener **una única responsabilidad** o motivo para cambiar. Promueve alta cohesión.

Patrón que aplica: *State, strategy, iterator, adapter*. Todos ellos se relacionan a una sola responsabilidad que se les delega desde otros objetos Contexto/Cliente.



“No debería haber nunca más de una razón para cambiar una clase”,

modelamos una clase, un concepto, que debería **concentrarse en hacer una sola cosa**, de forma que cambiar un requisito solamente se afecte a esa clase por una razón. Un módulo, clase o subsistema debería tener una única responsabilidad en el sistema totalmente encapsulada en sí mismo.

Responsabilidad: Obligación que tiene un objeto en cuanto a su comportamiento, donde implementamos métodos para llevarla a cabo, y que en consecuencia establece una razón para que una clase cambie. Por lo que, si hay más de una razón para que cambie significa que existe + de 1 responsabilidad. Relacionado con el concepto que esa clase modela, lo que define su responsabilidad.

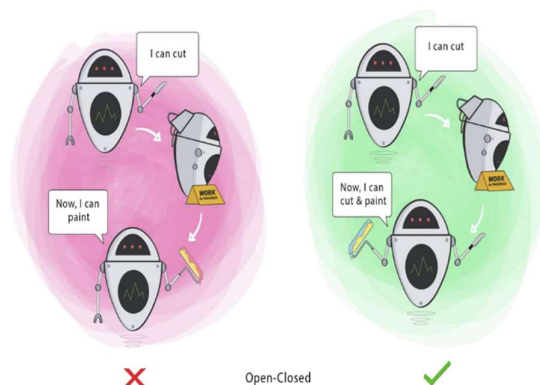
Permite alcanzar niveles de **bajo acoplamiento** y tener una **alta cohesión funcional**:

única responsabilidad + única razón para cambiar = bajo acoplamiento y alta cohesión.

Detectar si se viola el principio:

- ✓ En una misma clase hay 2 capas de la arquitectura.
- ✓ La cantidad de métodos públicos de la clase.
- ✓ Los métodos que usa cada atributo de la clase.
- ✓ Clase difícil de testear.
- ✓ Al incorporar una nueva funcionalidad, la clase se ve afectada.

Open/Closed Principle (OCP): Las entidades de software deben estar abiertas para **extensión**, pero **cerradas para modificación** de su funcionalidad.



Patrón que aplica: *State, strategy, observer, adapter, template Method*

Cerrado para agregar más métodos (para mantener SRP) pero abierto a usar mediante interfaces(extensión) otras clases para cumplir cierta funcionalidad. **[Permitir el cambio + No modificar lo existente = Flexibilidad]**

Ante la necesidad de un cambio no debería cambiar el SW existente, sino agregar algo nuevo que trabaje con el

anterior sin modificar su funcionamiento, esto se posibilita implementando los cambios mediante **herencia, polimorfismo y composición** (interfaces, clases abstractas). Extender significa agregar clases nuevas: permitir el cambio + no modificar lo existente = flexibilidad.

Detectar la violación del patrón:

- ✓ Clases que se modifican con frecuencia.

- ✓ Para cada requisito nuevo o cambio en los existentes, se afectan las mismas clases.

Patrón que aplica: State, strategy, observer, adapter, template Method. Estos patrones se aplican con herencia o realización, de manera que en la mayoría de los casos se pueden extender para incorporar funcionalidad extra (un nuevo estado en State, una nueva estrategia en Strategy, incorporar observadores nuevos en observer mediante la Interfaz Observador, un nuevo objeto adaptable con su correspondiente adaptador, o una nueva clase que herede de template e implemente su método plantilla).

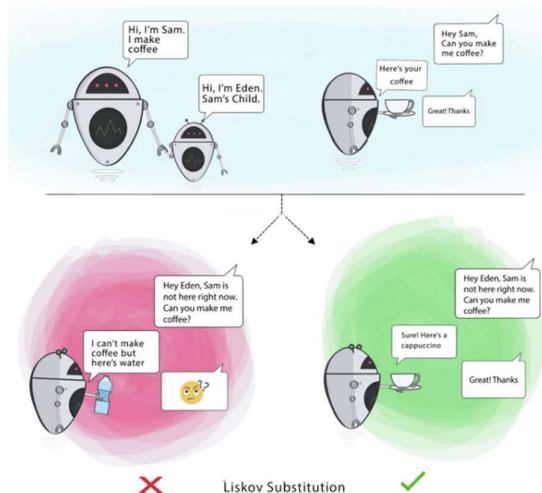
Liskov Substitution Principle (LSP): Los objetos de una clase hija deben poder sustituir a los objetos de la clase padre sin alterar el funcionamiento del programa. Asegura que una clase hija pueda reemplazar a su clase padre sin que el comportamiento del programa se vea afectado negativamente. Las funciones que usen o referencien clases base deberían poder usar objetos de subclases sin saberlo. Es una forma de control sobre el diseño, y busca **asegurar que los subtipos (hijos) cumplan correctamente con las interfaces propuestas por sus tipos base (padre)**, ya que sus **métodos reciben los mismos tipos de parámetros y tienen igual tipo de retorno**.

Patrón que aplica: State, Template method

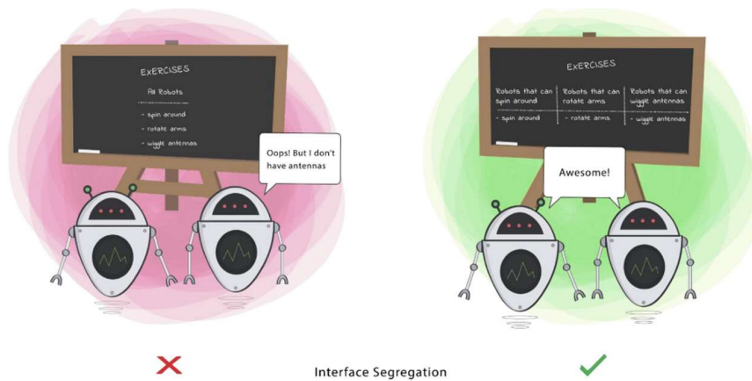
Habla de una **herencia bien diseñada**, para el reuso de comportamiento, surge para reutilizar código. **El beneficio** que nos provee la herencia **es la reutilización de comportamiento (código)**, la herencia es la **materialización del principio DRY** y reuso, de ser más efectivos, de ser más eficientes, trabajar menos. *La herencia sana es la **herencia de comportamiento no la de estructura**.*

¿Cómo detectar la violación del patrón?

- ✓ Un método reescrito por la clase hija lanza una excepción o no hace nada -> está violando el principio.
- ✓ Un método usado sobre algunas clases concretas ahora lanza una excepción -> no se puede asegurar que todas las clases siguen funcionando igual.
- ✓ Los test de la clase padre no funcionan para alguna clase hija.



Un **ejemplo** común para ilustrar el **LSP** es el "**problema del rectángulo y el cuadrado**". Si tenemos una clase `Rectángulo` con propiedades `ancho` y `alto`, y luego creamos una clase `Cuadrado` que hereda de `Rectángulo`, debemos asegurarnos de que las propiedades `ancho` y `alto` se comporten correctamente en ambas clases. Si modificamos el comportamiento en `Cuadrado` para que `ancho` y `alto` siempre sean iguales (ya que un cuadrado tiene lados iguales), estaríamos violando el LSP, ya que no podríamos sustituir un `Cuadrado` por un `Rectángulo` sin cambiar el resultado del programa.



Interface Segregation Principle

(ISP): Los clientes no deben verse obligados a depender de interfaces que no utilizan.

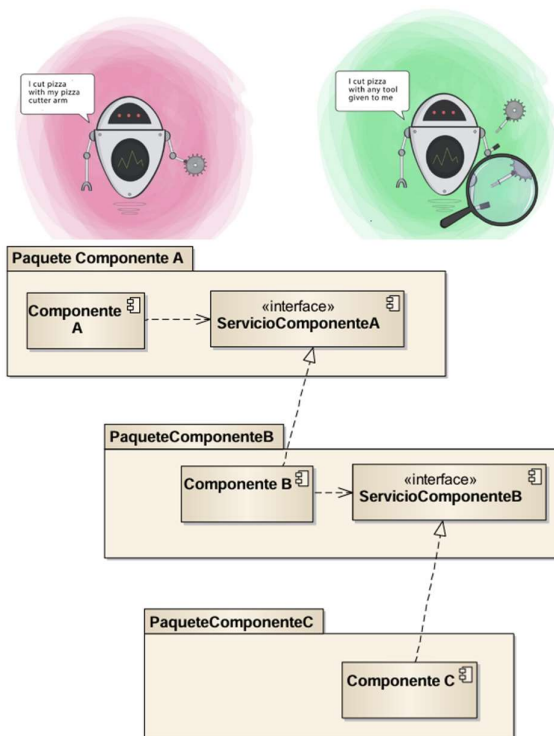
Las interfaces deben ser **específicas (cohesión)** y contener solo los métodos necesarios para evitar que las clases implementen métodos innecesarios.

Patrón que aplica: *Strategy, observer, adapter*

Una clase concreta puede implementar todas las interfaces que quiera. Al crear interfaces se tiene que asegurar que todas las clases que las implementen **necesiten y sean capaces de redefinir todos los métodos**. En caso contrario, es mejor separar en interfaces más pequeñas.

Problemática de las “fat interfaces”: cuando se definen interfaces con más responsabilidades de las debidas, las clases que la implementen no usarán muchos de esos métodos, y al estar obligados a re implementarlos, pueden definirse de modo que lancen excepciones, lo que puede hacer fallar el programa.

¿Cómo detectar si se viola el patrón? Al implementar una interfaz uno o varios de los métodos no tienen sentido para la clase, por lo que se dejan vacíos o tiran excepciones. Esto indica que se debe dividir la interfaz en otras específicas.



Dependency Inversion Principle (DIP):

Módulos de alto y bajo nivel: Los módulos de alto nivel no deben depender de los módulos de bajo nivel, ambos deben depender de abstracciones.

Abstracciones sobre detalles: Las abstracciones no deben depender de los detalles; son los detalles los que deben depender de las abstracciones (**INTERFACES**).

Flexibilidad, reutilización y robustez: se logra una mayor flexibilidad y robustez en el sistema. Los cambios en los módulos de bajo nivel no afectarán directamente a los módulos de alto nivel, ya que ambos se comunican a través de **interfaces abstractas**.

La inversión de dependencias permite que los **módulos** sean más móviles y reutilizables. Pueden **ser reemplazados o actualizados sin afectar la estructura general del sistema**.

Patrón que aplica: *Template Method*

En lugar de que los módulos de alto nivel dependan directamente de los módulos de bajo nivel, ambos deben depender de abstracciones. **Esto significa que los detalles específicos no deben ser la base de las relaciones entre los módulos.**

Se desacoplan ambas clases y se puede reemplazar una implementación de bajo nivel sin afectar a las clases base que la utilicen. (**Programar hacia la interfaz y no hacia la implementación**) Reduce el acoplamiento.

los módulos de alto nivel no dependen de los de menor nivel + ambos deben depender de sus abstracciones + las abstracciones no deben depender de los detalles= flexibilidad, robustez, movilidad.

Ej: La relación en el TM es del hijo al padre, en este caso para poder implementar la plantilla el padre apunta al hijo (invierte la dependencia). Se introduce este recurso para que funcione al revés de lo que funciona.

Ej: Esquema de persistencia: Cuando tengo comunicación entre capas, la relación natural es del cliente (presentación) hacia la base de datos (de arriba hacia abajo), en este caso para que el esquema funcione con un buen nivel de cohesión permitimos que el esquema de persistencia le haga una petición a la lógica de negocio (lo natural sería que la LN pida y la BD suba la rta)

Patrones de diseño GAMMA.

Patrón → Solución concreta a un problema bien identificado.

¿Qué es un patrón de diseño?

Son genéricos, **son soluciones habituales a problemas que ocurren con frecuencia** en el diseño de software. Cada patrón describe un problema que ocurre una y otra vez en un determinado entorno, así como la solución a ese problema.

Los patrones de diseño describen clases y objetos relacionados para resolver un problema de diseño general en un contexto. Para ello, **nominan, abstraen e identifican los aspectos claves de una estructura común** para que los diseños OO sean más flexibles, elegantes y reutilizables.

Los patrones de diseño son más puntuales que los arquitectónicos, **son de más bajo nivel**, rozan la implementación. En cada patrón se **identifican** las **clases/instancias participantes, roles, colaboraciones** y cómo éstas distribuyen sus responsabilidades. También aborda cuándo aplicarlo, las consecuencias, ventajas e inconvenientes que trae.

Patrones vs Idiomas

La diferencia de ambos es el alcance.

Un **patrón de diseño** tanto el problema como la solución son **suficientemente genéricos**, por lo tanto, son **independientes del lenguaje de programación**. En cambio, **el Idioma es un patrón de bajo nivel ya que es específico a un lenguaje de programación**. A estos patrones si se los adapta, presentan para un lenguaje de programación particular se los llama Idioma.

¿POR QUÉ USAR PATRONES? *Inducen a resolver todo tipo de problemas utilizando los principios del diseño orientado a objetos.*

- ✓ **Facilita la comunicación** en el equipo de trabajo, ya que con el nombre (si el equipo lo conoce) identificamos el problema a resolver.
- ✓ **Facilitan la reutilización** de buenos diseños y arquitecturas.
- ✓ **Nos ayudan a elegir alternativas de diseño** que hacen que un sistema sea reutilizable, y a evitar fallas que dificultan dicha reutilización.

- ✓ Pueden **mejorar la documentación y mantenimiento** de sistemas existentes.
- ✓ Ayudan a lograr un **buen diseño** rápidamente (mejoran la calidad de diseño).

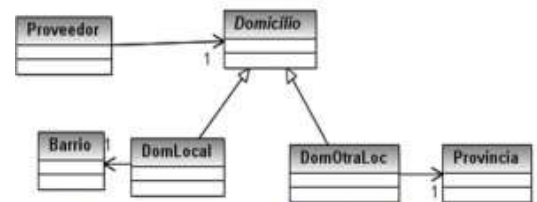
¿A QUE AYUDAN LOS PATRONES?

1. **Identificación de Objetos Apropriados:** Los patrones facilitan la **descomposición del sistema en objetos, mejorando el encapsulamiento, la flexibilidad y la reutilización**. Por ejemplo, patrones como *Strategy* y *State* ayudan a identificar abstracciones que no son obvias, permitiendo una alta cohesión y un bajo acoplamiento. Esto asegura que las clases del diseño se centren en la solución y se mantengan flexibles y extensibles.
2. **Definición de la Granularidad:** Deciden el **nivel de detalle en las responsabilidades de los objetos**. Patrones como *Facade* ayudan a representar subsistemas como un solo objeto, mientras que *Flyweight* permite la existencia de muchos objetos de granularidad fina, manteniendo la cohesión al reducir la responsabilidad de cada clase.
3. **Especificación de Interfaces:** Los patrones ayudan a **definir interfaces claras y manejables, asegurando que los objetos se comuniquen a través de ellas sin exponer su implementación**. Patrones como *Decorator* y *Proxy* fomentan el uso de interfaces que promueven el polimorfismo, permitiendo que los clientes interactúen con la interfaz y no con la implementación específica.
4. **Implementación de Objetos:** Los patrones **ofrecen pautas para definir las implementaciones mediante herencia y realización de interfaces**, ayudando a extender funcionalidades sin acoplamiento excesivo. ***Template Method* permite que una clase padre defina un algoritmo**, mientras que ***State* organiza el comportamiento mediante herencia y delegación**, adaptándose dinámicamente a distintos estados.

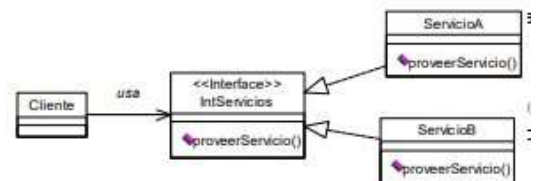
❖ **Herencia de clases (implementación):** permite **extender la funcionalidad** de un objeto **reutilizando la de una clase padre**. Esto facilita definir rápidamente nuevos tipos de objetos basados en otros, ya que **heredan el comportamiento necesario**.

❖ **Herencia de interfaces (Realizaciones de interfaz):** una interfaz define un **conjunto de características públicas para otros objetos**, separando la especificación de la implementación. Este mecanismo permite que un objeto se use en lugar de otro siempre que ambos implementen la misma interfaz.

Herencia de clases



Realizaciones de Interfaz



5. **Fomento de la Reutilización:** Al **priorizar la composición sobre la herencia, los patrones aumentan la flexibilidad**. Las dos técnicas más comunes para reutilizar funcionalidad en sistemas OO son el **reuso de caja blanca** (el interior de las clases padres se hace “visible” a las subclases) y **reuso de cajanegra** (los detalles internos de los objetos no son visibles), lo que se implementa mediante **herencia y composición** respectivamente.

La composición permite crear relaciones dinámicas en tiempo de ejecución, haciendo el diseño más adaptable, mientras que la herencia es más estática pero permite reutilizar estructuras de clases de forma rápida.

6. **Diseño para el Cambio:** La clave para maximizar la reutilización reside en anticipar nuevos requerimientos y cambios en los existentes, y en diseñar los sistemas de manera que puedan evolucionar en consecuencia. Esto garantiza que el sistema pueda escalar sin necesidad de un rediseño completo, haciendo que cada parte del sistema se pueda modificar o extender sin afectar significativamente a los demás componentes.

Causas comunes del rediseño: establecen los problemas comunes a los que se avoca cada patrón.

Crear un objeto indicando la clase.	Abstract factory, Factory Method, Prototype
Dependencia de operaciones específicas	Chain of Responsibility, Command
Dependencia de plataformas Hw o Sw	Abstract Factory, Bridge
Dependencia sobre representación de objetos.	Abstract factory, Bridge, Memento, Proxy
Dependencias de algoritmos	Builder, Iterator, Strategy, Template Method, Visitor
Acoplamiento fuerte entre clases	Abstract factory, Bridge, CoR, Command, Facade, Mediator, Observer
Extender funcionalidad mediante subclases	Bridge, CoR, Composite, Decorator, Observer, Strategy, State
Incapacidad de cambiar clases convenientemente	Adapter, Decorator, Visitor

¿Cómo se componen los patrones descritos a continuación?

1. **Nombre:** describe con una o dos palabras un problema de diseño junto con sus soluciones y consecuencias.
2. **Problema:** describe cuándo aplicar el patrón. Explica el problema y su contexto.
3. **Solución:** describe los elementos que forman el diseño, sus relaciones, responsabilidades y colaboraciones. Es como una plantilla que puede aplicarse en muchas situaciones diferentes.
4. **Consecuencias:** son los resultados, las ventajas e inconvenientes de aplicar el patrón. Son críticas para evaluar el patrón y entender los costos y beneficios de aplicarlo.
5. **Aplicabilidad.** Para que o cuando conviene aplicar dicho patrón.
6. **Implementación.** Serie de pasos que de forma generalizada explican implementación en casos prácticos.

Una de las **consecuencias que más se repite** de aplicar patrones, es que como **ventaja** obtenemos **Alta Cohesión** (Cohesión funcional) cada clase hace una sola cosa (principio Single responsibility - Responsabilidad Única) y como desventaja **se agregan más clases por ende crece la estructura**.

Los patrones marcados son los que se ven en Práctico. A medida que se vayan detallando serán remarcados de alguna forma notoria (son los más importantes).

A M B I T O		PROPÓSITO		
		CREACION	ESTRUCTURAL	COMPORTAMIENTO
		Factory Method	Adapter (de clase)	Interpreter <i>Template Method</i>
	CLASE <i>Estáticos</i>			
	OBJETO <i>Dinámicos</i> (es más flexible a diferencia de los patrones que trabajan con clases)	Abstract Factory Builder Prototype Singleton	Adapter (de objetos) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

SEGÚN EL PROPÓSITO: reflejan que hace el patrón.

Creación: Proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente.

Abstraen el proceso de creación de objetos, ayudando a crear **sistemas independientes de cómo los objetos son creados**, compuestos y representados, mejorando este proceso con la **distribución de responsabilidades**. Encapsulan las clases concretas que usa el sistema y cómo se crean sus instancias.

El sistema sólo conoce las clases abstractas, se separa la clase que invoca el constructor, proporcionando flexibilidad en qué, cómo, quién y cuándo al crear objetos.

- ❖ **Clases:** usa **herencia** para variar la clase instanciada.
- ❖ **Objetos:** usa **delegación** para instanciar otro objeto.

Estructurales: explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que se mantiene la flexibilidad y eficiencia de la estructura.

Están relacionados a cómo las clases y los objetos se combinan formando estructuras más complejas, manteniendo la flexibilidad y la eficiencia, y describiendo **formas** efectivas de **separar y combinar los elementos de la aplicación**.

Permiten añadir funcionalidad a objetos; simplificar estructuras complejas resolviendo necesidades puntuales de comportamiento en los objetos de forma dinámica, usando relaciones que se trasladen a los objetos; y lograr **compatibilidad** entre **clases** incompatibles de forma que cuando necesitamos que una clase acceda a servicios de otra, cuyas interfaces sean incompatibles, resolverlo con indirección (la creación de una **clase en el medio** de estas, que haga de **intermediario** y compatibilizando su comunicación).

- ❖ **Clases:** emplean **herencia** para definir las estructuras.
- ❖ **Objetos:** describen formas de combinar objetos obteniendo nueva funcionalidad, usando composición para posibilitar el cambio de los objetos en tiempo de ejecución.

Comportamiento: se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.

Se relacionan a **definiciones de algoritmos y asignación de responsabilidades a las clases**, enfatizando las **colaboraciones entre objetos** para llevar a cabo tareas más complejas, **reduciendo las dependencias entre ellos**.

Permiten asociar comportamiento a los objetos, invocar su ejecución, y abstraer secuencias de métodos.

- ❖ **Clases:** usan la **herencia** para distribuir el comportamiento.
- ❖ **Objetos:** usan la **composición** de objetos para **abstraer el comportamiento**.

PATRÓN STATE.

Ámbito→OBJETO | Propósito→COMPORTAMIENTO

Problema: el **comportamiento de un objeto depende de su estado**, sus métodos tienen lógica CASE que reflejan las acciones dependientes del estado.

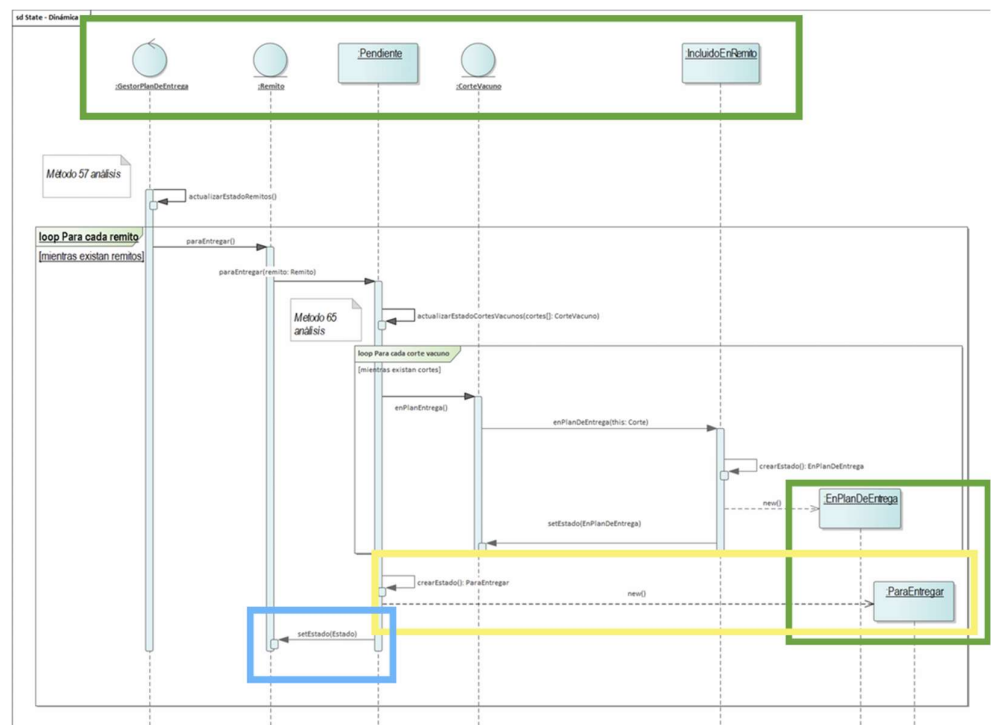
Se requiere que un objeto tenga **≠ comportamientos según el estado**, resultando complicado manejar el cambio de comportamiento y los estados en un mismo bloque de código.

Propósito: permite que un **objeto cambie su comportamiento cada vez que cambie su estado** interno.

Solución: crear clases “estado” por c/ estado posible, que implementen (o hereden) de una **interfaz/clase común (Estado abstracto)**. En lugar de definir en el contexto las operaciones que dependen del estado, se delegan en su objeto estado concreto actual.

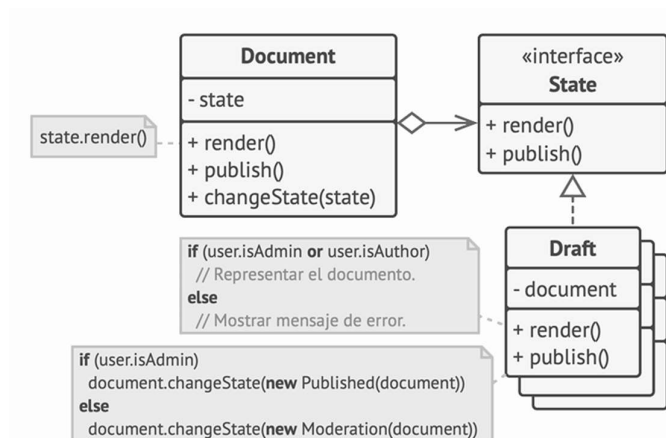
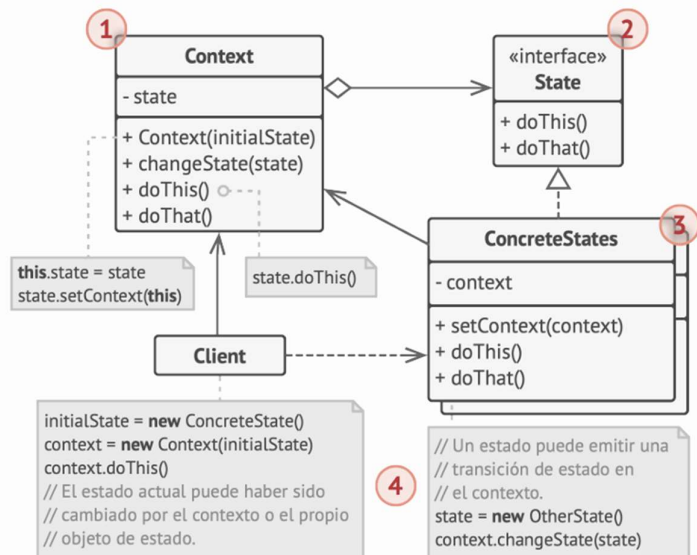
Implementación.

- Definir en qué parte del CU se va a aplicar el patrón
- Definir la clase Estado como abstracta con todos los métodos
- Definir los estados concretos
- Delegar al estado concreto la responsabilidad identificada
- Revisar consistencias entre estática y dinámica



Participantes:

1. **Contexto:** contiene los métodos que involucran las cuestiones vinculadas al estado.
2. **Estado abstracto (Interfaz):** define los métodos abstractos que invoca el Contexto al recibir una petición relacionada a su estado actual, abstrae al contexto del Estado concreto que responderá a su petición.
3. **Estados concretos:** son las subclases que heredan del Estado abstracto, resolviendo la petición del contexto en función de dicho estado concreto. Cada uno de ellos implementan los métodos definidos en la clase abstracta.
4. **Referencia Inversa:** Los objetos de estado pueden almacenar una referencia inversa al objeto de contexto. Esta referencia permite al estado extraer información del contexto e iniciar transiciones de estado.



Esta estructura puede resultar **similar al patrón Strategy**, pero hay una **diferencia** clave. En el patrón State, los **estados particulares pueden conocerse entre sí** e iniciar transiciones de un estado a otro, mientras que las **estrategias casi nunca se conocen**.

Consecuencias:

- Se coloca el comportamiento asociado al estado particular en una única clase.
- Evita estados inconsistentes.
- Transiciones de estado explícitas.
- Permite añadir nuevos estados y transiciones, agregando subclases.
- Incrementa el número de objetos, para “solucionar” o “aprovechar” esta situación se podrían compartir estados concretos entre varios objetos Contexto, en el caso de que no tengan atributos.
- Los objetos estado pueden implementarse como Singleton.

El contexto delega la responsabilidad a la clase Estado abstracto (invoca el mismo comportamiento en la clase Estado), se relaciona con la abstracción e instancia un objeto concreto. El estado

concreto es el que hace reutilización/redefinición de código de la clase Estado abstracta, **State funciona con Herencia.**

Un método de estado en la clase contexto (ej cancelar ()) invoca el comportamiento que está definido en la clase estado con un método (estado.cancelar()), pasándose a sí mismo (el objeto contexto) como parámetro para saber su estado actual (apuntando a un estado concreto). Si no se pasa a sí mismo como parámetro le tiene que pasar la información al estado para que lo pueda resolver. Le delega la responsabilidad al objeto en el que se encuentra (estado actual).

Aplicabilidad.

El comportamiento del objeto depende de su estado, debiendo cambiar en tiempo de ejecución dependiendo del estado. Las operaciones del objeto tienen grandes estructuras de tipo CASE (if, switch, etc.) que dependen del estado. El patrón State pone cada rama condicional en una clase estado concreta, resolviéndolo a través del polimorfismo (eliminando estructuras CASE / SWITCH en el código).

Principios de diseño OO se aplican en el patrón State ()

SRP: se reduce la responsabilidad en el Contexto, cada Estado Concreto mantiene una sola responsabilidad **OCP:** si se necesita cambiar se agregan nuevos estados Concretos (se extiende) para manejar la variabilidad y comportamiento. Se crea una jerarquía de clases nuevas para no modificar la clase contexto.

LSP: los Concretos se pueden intercambiar con el Abstracto, ya que todos son capaces de responder a sus métodos.

DRY: el código común se mantiene en la implementación trivial del padre, evitando redefinirlo en todas las hijas.

EWV: los Concretos encapsulan la lógica específica de cada uno, aislando los cambios del resto, sin afectarlos.

TDA: el Contexto delega el trabajo al Estado, que resolverá la lógica del pedido.

RELACIÓN CON OTROS PATRONES.

Bridge, State, Strategy (y, hasta cierto punto, Adapter) tienen **estructuras muy similares**. De hecho, todos estos patrones **se basan en la composición**, que consiste en delegar trabajo a otros objetos. Sin embargo, **todos ellos solucionan problemas diferentes**.

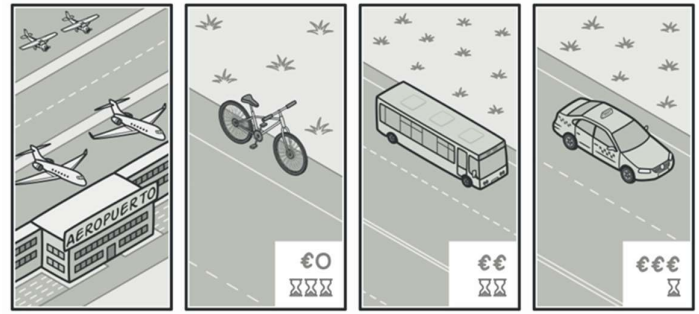
State puede considerarse una extensión de Strategy. Ambos patrones **se basan en la composición:** cambian el comportamiento del contexto delegando parte del trabajo a objetos ayudantes. **Strategy hace que estos objetos sean completamente independientes y no se conozcan entre sí.** Sin embargo, State no restringe las dependencias entre estados concretos, permitiéndoles alterar el estado del contexto a voluntad.

PATRÓN STRATEGY.

Ámbito→OBJETO

Propósito→COMPORTAMIENTO

Problema: existencia de **muchos algoritmos en una sola clase**, por ejemplo una clase que según el tipo de cliente, realice distintas funcionalidades.



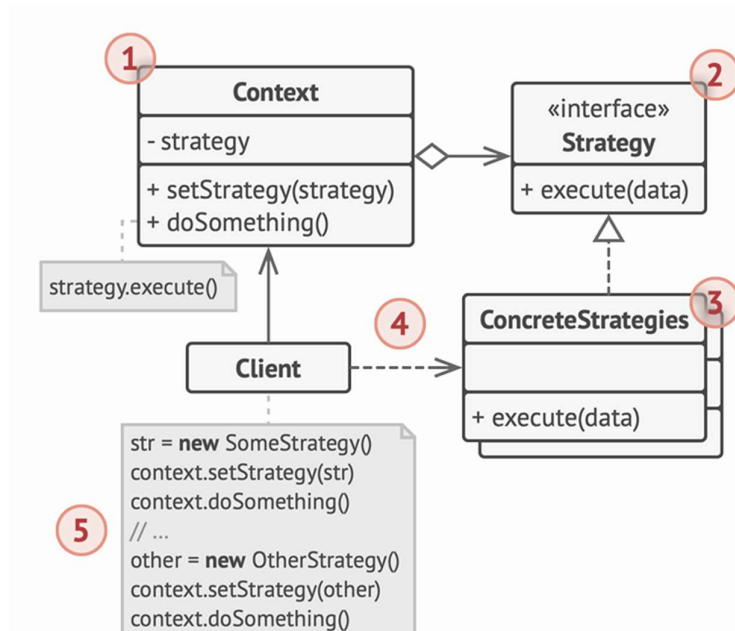
Varias estrategias para llegar al aeropuerto.

Propósito: Define una familia de algoritmos, encapsulando cada uno de ellos en clases, y permite elegir cuál estrategia utilizar en tiempo de ejecución. Permite variar los algoritmos independientemente de los clientes que los usen

Solución: Sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas estrategias.

La clase original, llamada contexto, debe tener un campo para almacenar una referencia a una de las estrategias. El contexto delega el trabajo a un objeto de estrategia vinculado en lugar de ejecutarlo por su cuenta. El cliente pasa la estrategia deseada a la clase contexto. De hecho, la clase contexto no sabe mucho acerca de las estrategias. Funciona con todas las estrategias a través de la misma interfaz genérica, que sólo expone un único método para disparar el algoritmo encapsulado dentro de la estrategia seleccionada

Participantes:



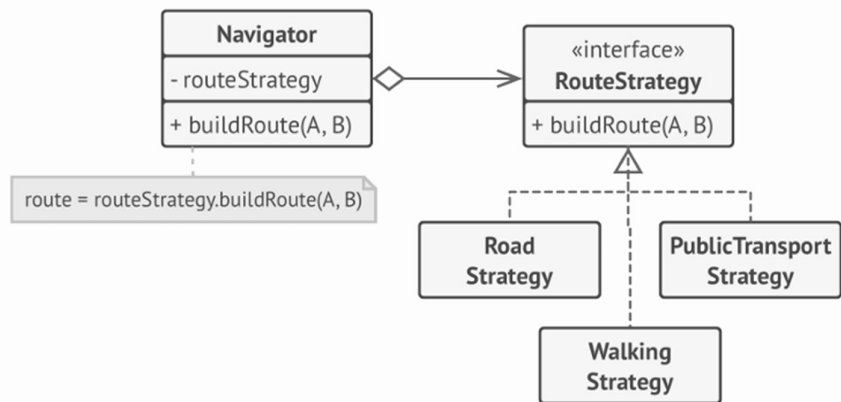
1. Contexto: es la clase que **utiliza** las diferentes estrategias (**algoritmos**) para alguna tarea, **mantiene** referencias a 1 o + **instancias de Estrategia concreta**, **configurables para el uso en tiempo de ejecución**. Tiene la responsabilidad de configurar la estrategia concreta a usar antes de invocar al método del algoritmo (pero mantiene una referencia a la Interfaz), y de la ejecución del método polimórfico. Delega la operación algorítmica a una clase concreta sin saber cuál.

2. Estrategia (Interfaz/Abstracta): declara una interfaz común a todos los algoritmos permitidos, define la signatura de los métodos disponibles para que el Contexto utilice.

3. Estrategia Concreta: implementan cada uno de los algoritmos variantes de las operaciones definidas en la interfaz Estrategia.

4. La clase contexto invoca el método de ejecución en el objeto de estrategia vinculado cada vez que necesita ejecutar el algoritmo. La clase contexto no sabe con qué tipo de estrategia funciona o cómo se ejecuta el algoritmo.

5. El Cliente crea un objeto de estrategia específico y lo pasa a la clase contexto. La clase contexto expone un modificador set que permite a los clientes sustituir la estrategia asociada al contexto durante el tiempo de ejecución.



Aplicabilidad.

Se usa para configurar una clase con 1 o + comportamientos posibles, cuando son necesarias ≠ variantes de un algoritmo, o una clase que define muchos comportamientos como sentencias CASE dentro de sus métodos.

Consecuencias:

- Define una familia de algoritmos relacionados.
- Es una alternativa a crear subclases de la clase Contexto.
- Elimina sentencias CASE.
- El cliente puede elegir entre diferentes estrategias o implementaciones, debiendo conocer sus detalles.
- Se incrementa el número de objetos (Se puede resolver este problema con un patrón Flyweight).

Solución en detalle: Se puede resolver con herencia y realización. El contexto recibe la información sobre la estrategia específica a usar (actor que selecciona algo, por parámetro, etc.), el contexto puede tener más de una estrategia al mismo tiempo en la estructura (1 a *).



El contexto le delega a la estrategia. Con herencia cada estrategia concreta implementa lo que heredó de la estrategia abstracta.

Con **realización** las estrategias concretas implementan cada una el comportamiento

polimórfico que está declarado en la interfaz. En este caso las clases concretas están obligadas a redefinir el comportamiento. Lo usual es utilizar

realización, ya que cada estrategia concreta es ≠ a las demás y no hay reutilización de código.



El contexto crea la interfaz concreta con un new() y se pasa como parámetro a sí mismo para otorgarle visibilidad y que la estrategia obtenga la información, o se la pasa como parámetro al llamar el método polimórfico para que la estrategia resuelva el problema.

```
contexto.setStrategy(new EstrategiaConcreta(this))
```

El contexto puede actuar redirigiendo peticiones de otros objetos clientes hacia la estrategia. La delegación está en que el contexto tiene una invocación al método de la Estrategia.

Relacion con otros patrones y el Strategy.

TemplateMethod se basa en la herencia: te permite alterar partes de un algoritmo extendiendo esas partes en subclases. Strategy se basa en la composición: puedes alterar partes del comportamiento del objeto suministrándole distintas estrategias que se correspondan con ese comportamiento. **Template Method trabaja al nivel de la clase, por lo que es estático.** **Strategy trabaja al nivel del objeto, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.**

Principios de diseño OO que se aplican en el Strategy:

OCP: el Contexto está abierto para extensión, ya que se pueden agregar más Estrategias sin cambiarlo.

ISP: les da a las clases concretas los métodos que necesita implementar, cada estrategia es específica (alta cohesión).

DRY: la parte común del algoritmo se mantiene en el contexto, evitando duplicaciones. El contexto tiene la llamada del método, a través de la delegación el código a ejecutar lo tiene la estrategia que lo resuelve. **ECV:** la parte que varía del algoritmo se encapsula en las Estrategias. Ej.: cálculos del IVA (puede cambiar el IVA por estrategia)

PTI: se establece una interfaz única y todas las Estrategias se implementan hacia ella para utilizarse en el Contexto.

COI: si se usa composición para que el contexto mantenga referencias a las Estrategias, en vez de una clase base con el comportamiento general e hijas con la parte que cambia. Se aplica delegación en el contexto y estrategia, y se reutiliza código al solo hacer la llamada.

PATRÓN TEMPLATE METHOD.

Ámbito → CLASE | Propósito → COMPORTAMIENTO

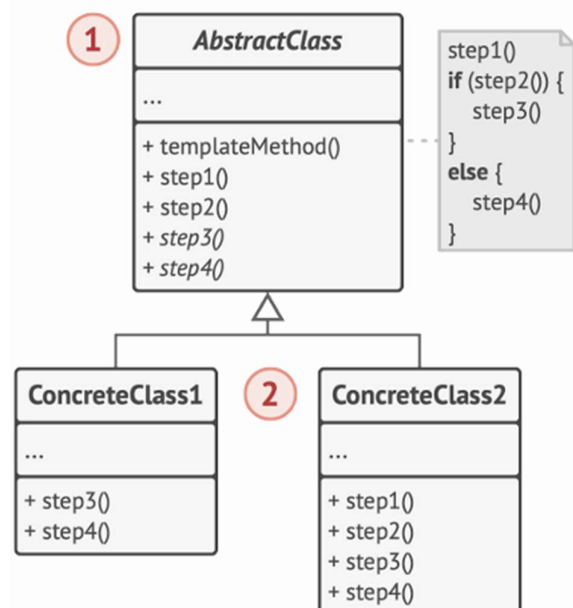
Problema: Varias subclases implementan algoritmos similares, pero con diferencias menores.

Propósito: Define el esqueleto de un algoritmo en la superclase pero permite que las subclases sobrescriban pasos del algoritmo sin cambiar su estructura.

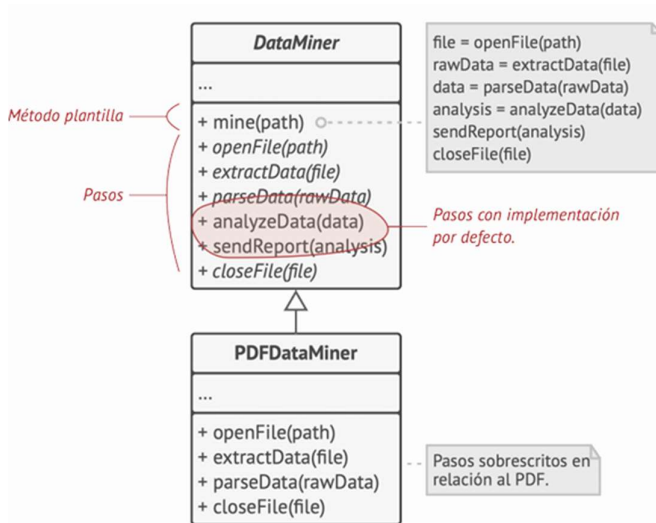
Solución: Define la estructura de un algoritmo en una clase base y permite que las subclases implementen pasos específicos.

Participantes:

1. Clase Abstracta: implementa un método plantilla que define el esqueleto de un algoritmo, con operaciones primitivas, fijando el orden de las operaciones abstractas, que pueden o no ser redefinidas por las subclases al implementarlo.



2. Clase Concreta: implementa las operaciones primitivas para realizar los pasos del algoritmo, sobre escribiendo los métodos abstractos de la superclase.



Motivación: una clase aplicación que maneja objetos de clase documento, se usa el método **openDocument** para guardar documentos en **distintos formatos**, cambiando algorítmicamente solo lo del formato específico.

Aplicabilidad: se utiliza en **estructuras con Herencia**, cuando es **necesario implementar partes fijas en un algoritmo**, y que las subclases implementen el comportamiento variable, y al revés, cuando el comportamiento común entre subclases tiene que ser factorizado y localizado en una superclase. O cuando se necesitan controlar extensiones de las subclases.

Este patrón se usa para **garantizar** que **todas** las subclases que **van a heredar** el método plantilla implementen **el algoritmo** que está contenido, **en el orden de invocación de las operaciones** que está definido en el método. El **orden de ejecución** de las operaciones del método plantilla **es el mismo para todas** las subclases, **cada subclase puede redefinir un paso de forma distinta**, que se llamará **método enganche**. Normalmente, los enganches se colocan antes y después de pasos cruciales de los algoritmos, suministrando a las subclases puntos adicionales de extensión para un algoritmo.

El método plantilla **se define en la clase PADRE** como **público final**, para que las **hijas** no lo puedan modificar, pero sí **modificar polimórficamente las operaciones que se invocan desde el método plantilla**. SIEMPRE debe tener implementación.

Un método plantilla (template) invoca los siguientes tipos de métodos:

- ❖ **Operaciones abstractas del padre:** obligan a las subclases a redefinirlas.
- ❖ **Operaciones concretas del padre:** tienen implementación por defecto en la superclase.
- ❖ Operaciones concretas sobre otras clases Cliente.
- ❖ **Método enganche:** lo tiene definido la clase padre para que todos los hijos lo puedan usar, ignorar, o redefinir en función de lo que necesiten (puede tener comportamiento por defecto o ser vacío).

Beneficios:

- ✓ **Evita que las subclases deban redefinir el método completo** de la clase padre, cada método que sea igual en todas se define en la clase base como método plantilla.
- ✓ **Evita la duplicación de código** en la definición del **método polimórfico** para las subclases, fundamental para la **reutilización del mismo**.

Consecuencia: Puede llevar a jerarquías de clases rígidas.

Estructura de control invertida, relacionada con el principio Inversión de dependencia.

Las invocaciones en la herencia son del hijo al padre, principalmente los hijos invocan operaciones del padre, se invierte el orden natural. Se plantea que dentro del metodoPlantilla() que está en la clase PADRE, tengo invocaciones a las operaciones concretas de las clases hijas.

Principio de Hollywood (No nos llamen, nosotros te llamamos) → La clase padre llama a la clase hija. Se da en compilación (t.ejecucion).

En términos de objetos, la clase concreta (Hija) es instanciada y hereda el método plantilla, invocando la operación a sí misma. Ya que la **clase abstracta no se puede instanciar**.

En **términos de estructura** estamos **aplicando la inversión de dependencia**, porque estamos haciendo una invocación de un método de la clase hijo en el método plantilla que está en el padre.

Principios de diseño OO se aplican en el patrón Template Method()

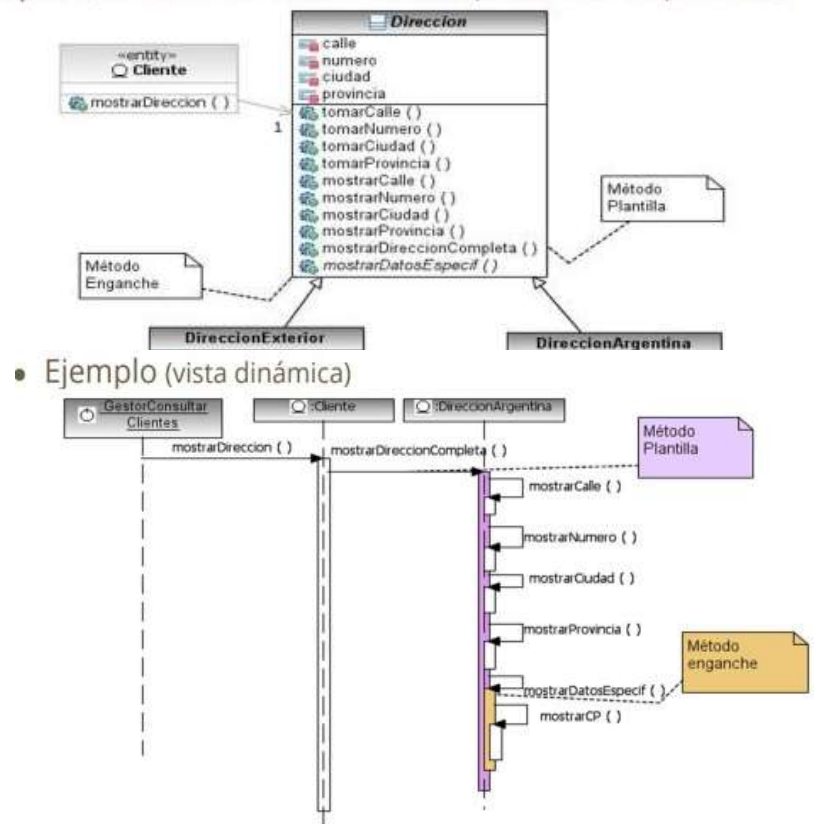
LSP: Analiza si está bien construida la herencia, ya que se puede invocar cualquier método plantilla en las subclases y se van acomodar en el mismo orden que la clase padre, con operaciones definidas en ella.

DIP: principio de inversión de dependencia. Le da más potencia al método plantilla y que permite que en el método plantilla haya invocaciones a sus clases hijas (naturalmente la herencia no lo permite), el padre apunta al hijo y en herencia es al revés. Permite que el TM pueda armar una plantilla completa, con invocaciones a clases concretas.

Relaciones con otros patrones.

- **Factory Method es una especialización del Template Method.** Al mismo tiempo, un Factory Method puede servir como paso de un gran Template Method.
- **TemplateMethod se basa en la herencia:** te permite alterar partes de un algoritmo extendiendo esas partes en subclases. **Strategy se basa en la composición:** puedes alterar partes del comportamiento del objeto suministrándole distintas estrategias que se correspondan con ese comportamiento.
- **Template Method trabaja al nivel de la clase**, por lo que es estático. **Strategy trabaja al nivel del objeto**, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.

Ejemplo (vista de la estructura aplicando el patrón)



PATRÓN OBSERVER

(publish-suscribe).

Ámbito→OBJETO

Propósito→COMPORTAMIENTO

Problema: Cuando el estado de un objeto cambia, muchos otros objetos deben ser **notificados** sin un fuerte acoplamiento.

Un **mecanismo de suscripción** para **notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando**.

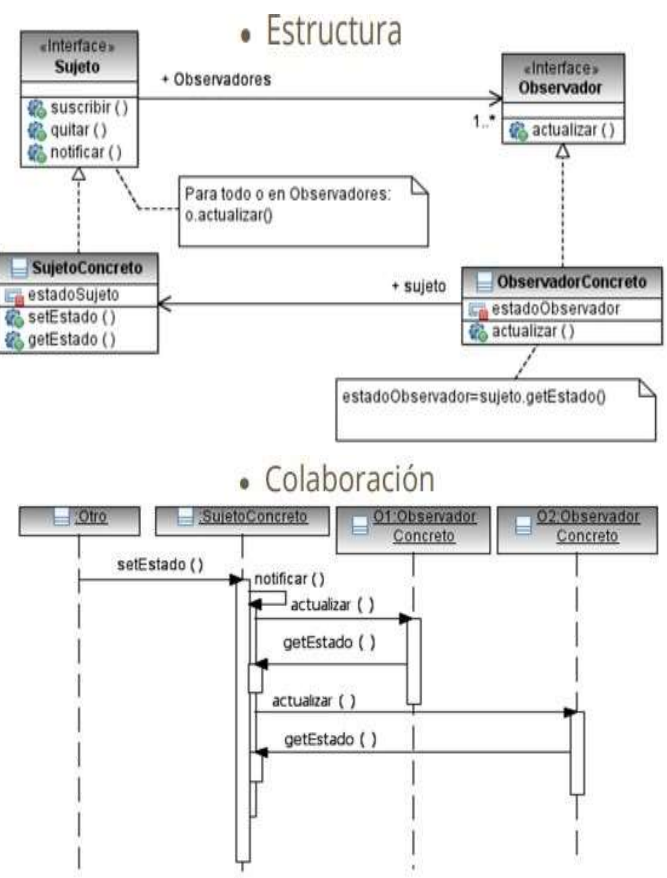
Propósito: Define una **dependencia uno-a-muchos entre objetos**, de modo que **cuando cambia el estado de un objeto**, todos **sus dependientes** automáticamente **son notificados** y actualizados.

Solución: El **objeto que tiene un estado interesante suele denominarse sujeto**, pero, como **también va a notificar** a otros objetos los cambios en su estado, le **llamaremos notificador o publicador**. El **resto** de los objetos que quieren conocer los cambios en el estado del notificador, **se denominan suscriptores**

El notificador mantiene una **lista de suscriptores** y sabe qué revistas les interesan. Los **suscriptores pueden abandonar la lista en cualquier momento si quieren** que el notificador deje de enviarles nuevos números. Es fundamental que todos los **suscriptores implementen la misma interfaz** y que **el notificador únicamente se comunique con ellos a través de esa interfaz**.

Participantes:

- **Sujeto:** conoce a sus objetos observadores. Es una interfaz que proporciona métodos para asignar y quitar objetos Observador.
- **Observador:** define una interfaz para actualizar los objetos que deben notificarse ante cambios en un Sujeto.
- **SujetoConcreto:** implementa la interfaz Sujeto, y almacena el estado que es de interés a notificar en los Observadores concretos. Cuando cambia su estado, notifica a los observadores.
- **ObservadorConcreto:** implementa la interfaz de actualización del Observador, manteniendo su estado consistente cada vez que el Sujeto cambie su estado. Mantiene referencia con un objeto SujetoConcreto.



Aplicabilidad.

El **cambio de estado en un objeto requiere cambios en otros**, y no se sabe sobre qué objetos. O cuando **un objeto tiene que ser capaz de notificar algo a otros objetos sin importar quienes son**.

Consecuencias:

- Permite **modificar independientemente** los sujetos y observadores.
- **Acoplamiento abstracto y mínimo entre Sujeto y Observador**, por lo que pueden reutilizarse por separado, **añadiendo observadores sin modificar el sujeto**, ya que este último no necesita conocer las clases observadores concretas.
- **Soporte a la comunicación broadcast** (COM: multipunto-PMP) en la que el **Sujeto no especifica al receptor** concreto de la notificación, **pudiendo añadir/eliminar** observadores en **cualquier instante**.

Implementacion.

- La estructura puede ser con **Herencia** o **Realización**.
- Se puede aplicar este patrón con la forma **Pull: se notifica y actualiza a los observadores cuando éstos lo desean**, el sujeto se envía a sí mismo como parámetro en las notificaciones para que el observador obtenga los datos que necesita.
O de la forma **Push**: sirve más para **cundo las notificaciones son constantes**, y el Sujeto que notifica envía todos los datos necesarios a los observadores.
- El Sujeto concreto se envía por parámetro también en los casos que los observadores están suscriptos a +1 Sujeto concreto.
- La notificación, por lo general, es disparada por métodos Set en la clase Sujeto o por los clientes de la clase Sujeto (otros objetos que se relacionan con él).
- Al registrar un observador también se le puede asociar el evento sobre el cual notificarlo: suscribir (obs: Observador, e: Evento).
- Cuando las relaciones entre Sujeto y Observador son complejas, **se pueden encapsular las actualizaciones usando el patrón Mediator**. Un uso de este patrón es en aplicaciones Modelo - Vista, donde el modelo tiene los datos y al modificarlos, tiene que notificar/actualizar a los observadores que son las vistas.

Principios de diseño OO se aplican en el patrón Observer.

Segregación de interfaces (ISP): se usan 2 interfaces, el comportamiento relacionado a los sujetos se define en la ISujeto, para que los sujetos concretos lo implementen, lo mismo se hace con los observadores.

Programar hacia la interfaz y no hacia la implementación (PTI): se reduce el acoplamiento entre sujetos y observadores, al comunicarse a través de la interfaz.

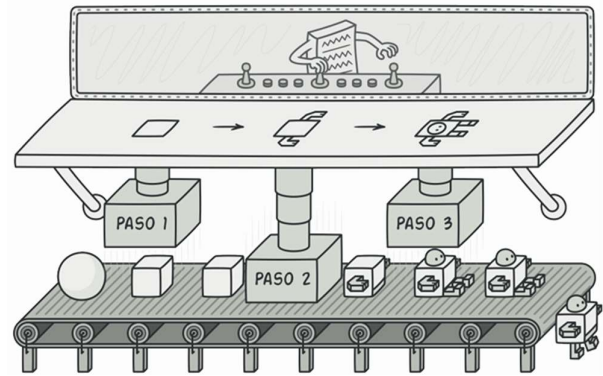
Abierto para la extensión y cerrado para la modificación (OCP): permite introducir nuevas clases observadores sin tener que modificar el sujeto (solamente implementando la interfaz de observadores) y viceversa.

PATRÓN BUILDER.

Ámbito→OBJETO.

Propósito→CREACION.

Problema: Gestionar la **construcción de objetos complejos con múltiples configuraciones es difícil** y puede llevar a constructores sobrecargados o numerosas subclases.



Propósito: Separa la **construcción de un objeto complejo** de su representación, así que **el mismo proceso** de construcción puede **crear diferentes representaciones**

Solución: Usar una clase **Builder** para construir el objeto paso a paso, permitiendo flexibilidad y control sobre el proceso de construcción.

*“Patrón de diseño **creacional** que nos permite **construir objetos complejos** paso a paso. El patrón nos permite **producir distintos tipos** y representaciones de un objeto **empleando el mismo código de construcción**”.*

Participantes:

Adawsd

Aplicabilidad:

- Cuando **el algoritmo para crear un objeto complejo debe ser independiente de las piezas que conforman el objeto** y de cómo se ensamblan.
- El proceso de construcción debe permitir **diferentes representaciones para el objeto que se construye**.

Consecuencias:

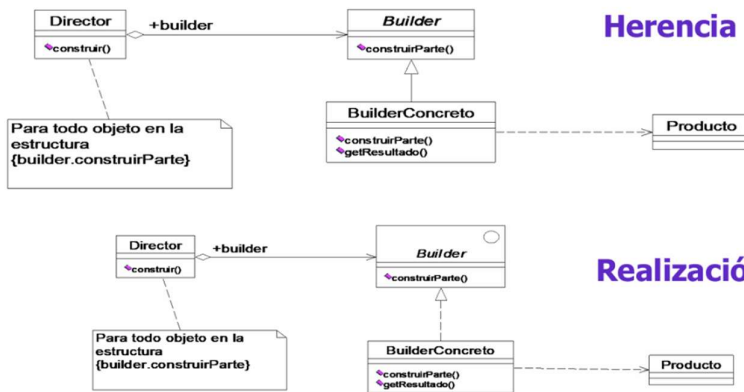
- Proporciona un **control fino del proceso de construcción**.
- **Separación de la construcción y la representación:** Permite cambiar la representación interna del producto sin afectar el código que lo construye.

Implementación:

- **Interfaz General:** La interfaz de **Builder** debe ser lo suficientemente **flexible** para permitir la construcción de diferentes tipos de productos por cualquier implementación concreta de Builder. Esto asegura que el patrón pueda ser utilizado en una variedad de contextos.
- **Construcción Compleja:** La construcción de un producto puede ser más complicada que simplemente añadir un nuevo componente. Puede involucrar múltiples pasos y configuraciones específicas.
- **Métodos Vacíos:** Los métodos en la clase **Builder** no son abstractos, sino que **están definidos como vacíos**. Esto permite que las subclases los implementen según sea necesario, proporcionando **flexibilidad** en la construcción del producto.

- **Clases de Productos:** Las clases de los productos contruidos no comparten una clase abstracta común. Esto significa que cada producto puede tener su propia estructura y características únicas, sin necesidad de heredar de una clase base común.

Participantes:



Constructor: Define una interfaz abstracta para crear las partes de un objeto complejo.

Constructor Concreto: Implementa la interfaz Constructor para construir y ensamblar las partes del objeto producto.

Director: Encargado de crear un objeto usando la interfaz Constructor.

Producto: Representa el objeto complejo que es construido.

Procedimiento:

- **Definir partes:** Identificar las diferentes partes que deben construirse para obtener el objeto completo.
- **Especificar interfaz Constructor:** Definir los métodos para construir cada una de las partes.
- **Establecer Director:** Clase responsable de dirigir, coordinar y controlar la construcción.
- **Crear Constructores Concretos:** Clases que implementan la interfaz Constructor para ensamblar los productos.

PATRÓN ITERATOR.

Ámbito→OBJETO.

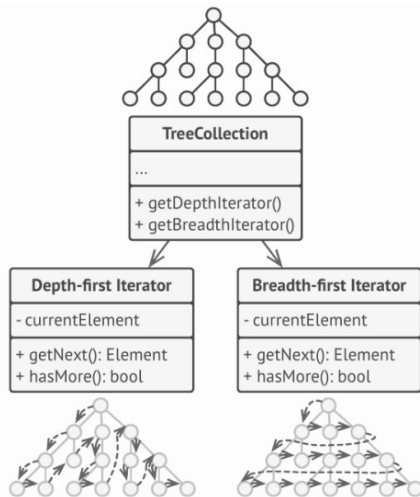
Propósito→COMPORTAMIENTO.

Problema: Navegar por los elementos de un conjunto sin exponer su representación subyacente es complicado y depende de su implementación.

Propósito: Proporciona una **forma de acceder a los elementos de un objeto** agregado de manera secuencial **sin exponer su representación interna**. Permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).



Solución: La idea central del patrón Iterador es **extraer el comportamiento de recorrido** de una colección y **colocarlo en** un objeto independiente llamado *iterador*. Usa una **interfaz Iterador** para **definir métodos para recorrer** los elementos, sin importar cómo se almacenan.



Los iteradores implementan varios algoritmos de recorrido. Varios objetos iteradores pueden recorrer la misma colección al mismo tiempo.

Motivación.

Un objeto agregado o una colección, debe permitir una forma de recorrer sus elementos sin exponer su representación interna, con diferentes algoritmos de recorrido, y de forma concurrente, sin añadir dicha funcionalidad a la definición de la colección.

Aplicabilidad.

Casos en los que se requiere el acceso secuencial a los contenidos de un objeto agregado, soportando múltiples recorridos. El **Iterador se generaliza con una interfaz**, para poder **soportar los recorridos de diferentes estructuras** agregadas (iteración polimórfica), y cambiarlas.

Consecuencias. Se **simplifica la interfaz del contenedor**, al extraer en Iterador los métodos de recorrido. Posibilita realizar **varios recorridos** concurrentes, y anidados, y **variando las técnicas para recorrer**.

Implementación. Al implementarlo se debe tener en cuenta quién controla la iteración:

Externo: un objeto **Cliente**, quien **instanció al Iterador**, controla el recorrido; **Interno:** el mismo **iterador controla sobre sí mismo la iteración**, devolviendo los elementos al objeto cliente según la lógica definida. También se debe definir quién establece el algoritmo de recorrido, si el Agregado (el Iterador solo almacena el estado de iteración) o el Iterador (posibilita reutilizar el mismo algoritmo en ≠ colecciones o aplicar ≠ algoritmos en una).

Participantes(Realizacion):

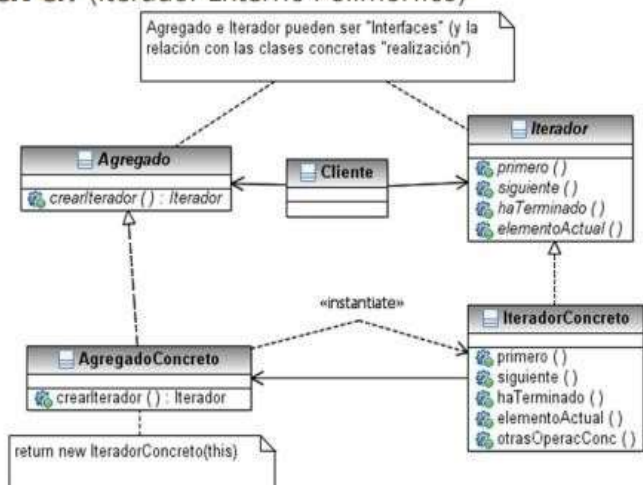
• Estructura: (Iterador Externo Polimórfico)

Iterador: es la **interfaz** que define las operaciones para recorrer y acceder a los elementos de la estructura de datos.

Iterador Concreto: *implementa Iterador*, manteniendo la posición actual en el recorrido del Agregado, sabiendo qué elementos ya recorrió.

Lista (Agregado): *define una interfaz* para la creación de un objeto Iterador.

Agregado Concreto: *implementa la interfaz de creación de Lista/Agregado* para devolver una instancia de Iterador Concreto.



Aplicación del Patrón Iterator(ejemplo).

1. Objetos Involucrados

- **Catalogo:** Es la clase que contiene los elementos que se desean recorrer (en este caso, probablemente una lista de secciones o artículos).
- **IteradorSeccion:** Representa el iterador concreto que permitirá recorrer las secciones del catálogo. Este iterador contiene métodos específicos para moverse a través de los elementos y obtener información sobre el estado de la iteración.

2. Inicio de la Iteración

- **Llamada a crearIterador():** El objeto Catalogo inicia la creación del iterador llamando a crearIterador(elementos: Object[]), que devuelve una instancia de IteradorSeccion. Este método es el encargado de crear y devolver un iterador adecuado para la estructura Catalogo.
- **Método primero():** Una vez que se ha creado el iterador, se llama a primero() en el IteradorSeccion, que posiciona el iterador en el primer elemento de la colección. Este es el primer paso del patrón Iterator, ya que establece el inicio del recorrido.

3. Recorrido de los Elementos

- **Bucle de Iteración (loop recorrer secciones):** En el diagrama, se utiliza un bucle controlado por la condición [haTerminado() := false]. Esto indica que el bucle seguirá ejecutándose mientras el iterador no haya alcanzado el final de la colección.
- **Método haTerminado():** En cada iteración, haTerminado() se llama para verificar si el iterador ha llegado al último elemento. Si devuelve false, la iteración continúa; de lo contrario, el bucle se detiene.
- **Método actual():** Dentro del bucle, actual() se invoca para obtener el elemento en la posición actual del iterador. Este método devuelve un objeto (en este caso, una sección del catálogo) sobre el cual el cliente puede operar.
- **Método siguiente():** Después de procesar el elemento actual, siguiente() se llama para avanzar al próximo elemento en la colección. Esto desplaza el iterador a la siguiente posición, y el ciclo se repite hasta que haTerminado() devuelva true.

4. Obtención de Información Específica del Elemento

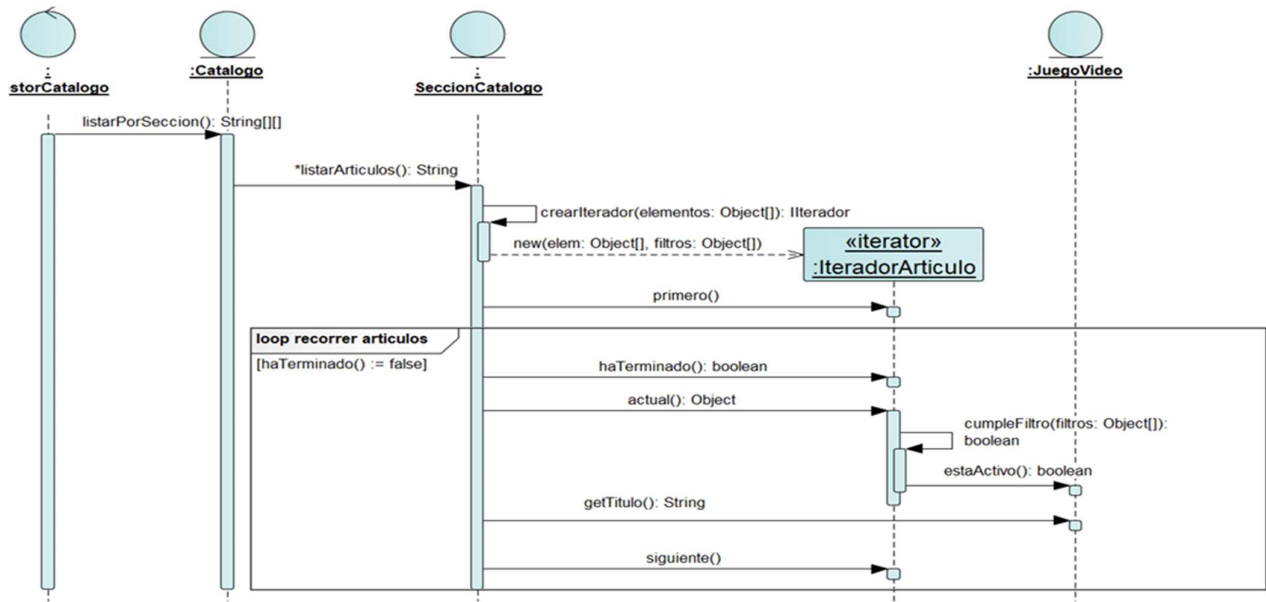
- **Método getNombre():** En el ejemplo, también se muestra el método getNombre() del iterador, que se usa para obtener el nombre o información específica del elemento actual. Este método ilustra cómo se puede interactuar con los elementos a medida que se recorren.

5. Resumen de la Secuencia

- El proceso comienza llamando a crearIterador() para obtener un iterador específico para Catalogo.
- primero() establece la posición inicial del iterador.
- haTerminado() verifica si aún hay elementos por recorrer.

- `actual()` proporciona el elemento actual y permite al cliente procesarlo.
- `siguiente()` mueve el iterador al siguiente elemento.
- Este ciclo continúa hasta que `haTerminado()` indique que se ha llegado al final de la colección.

En este ejemplo, el patrón Iterator permite recorrer los elementos de Catalogo de manera uniforme y secuencial sin exponer los detalles de la implementación interna de Catalogo. El iterador proporciona un conjunto de métodos estandarizados para manipular el recorrido (iniciar, avanzar, verificar fin) y abstrae al cliente de los detalles de la estructura de Catalogo. Así, el cliente puede interactuar con Catalogo sin tener que saber cómo están almacenadas las secciones internamente, solo necesita usar el iterador.



Principios de diseño OO se aplican en el patrón Iterator

Principio de responsabilidad única: Iterator separa de los objetos Cliente las responsabilidades de recorrer y acceder a los elementos de la estructura agregada, haciendo que sea la única responsabilidad del Iterator.

Principio de abierto/cerrado: permite implementar nuevas colecciones con sus iteradores y agregarlos al código de clases existentes sin descomponer ni modificar nada.

Principio de inversión de dependencias: permite que las clases Clientes dependan de abstracciones (las interfaces de Iterator y de Lista), en vez de implementaciones concretas, lo que permite sustituir distintos iteradores o colecciones sin afectarlas.

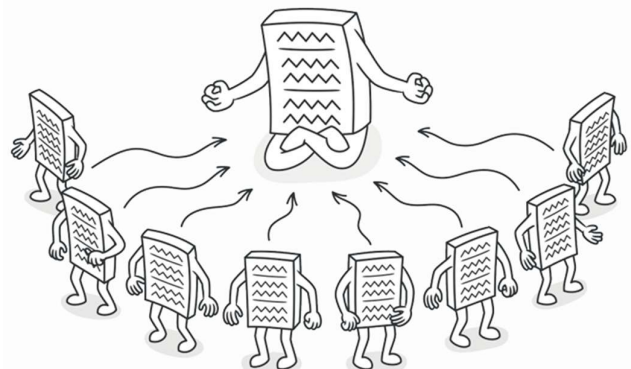
Principio Don't Repeat Yourself (DRY): la interfaz Iterator ayuda a no tener código duplicado relacionado a la iteración y navegación de las colecciones, encapsulado en esa única interfaz en lugar de repetirlo en cada estructura agregada.

PATRÓN SINGLETON.

Ámbito→OBJETO.

Propósito→CREACIÓN.

Problema: Asegurar que una clase tenga solo una instancia global puede ser complicado en entornos multi-hilo.



Propósito: Permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.

El patrón Singleton resuelve dos problemas al mismo tiempo, vulnerando el **Principio de responsabilidad única**:

1. **Garantizar que una clase tenga una única instancia.** ¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es **controlar el acceso a algún recurso compartido**, por ejemplo, una **base de datos** o un archivo.
Ten en cuenta que este comportamiento es imposible de implementar con un constructor normal, ya que una llamada al constructor siempre debe devolver un nuevo objeto por diseño.
2. **Proporcionar un punto de acceso global a dicha instancia.** ¿Recuerdas esas variables globales que utilizaste (bueno, sí, fui yo) para almacenar objetos esenciales? Aunque son muy útiles, también **son poco seguras**, ya que **cualquier código podría sobrescribir** el contenido de esas variables y descomponer la aplicación.

Al igual que una variable global, el **patrón Singleton nos permite acceder a un objeto desde cualquier parte del programa**. No obstante, **también evita que otro código sobrescriba esa instancia**.

Solución: Proporciona un método de clase que crea o retorna la única instancia de la clase. Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador *new* con la clase Singleton. Crear un método de creación estático que actúe como constructor.

• Estructura



```

public static Singleton getInstancia()
{ if (unicInstancia == null)
  unicInstancia = new Singleton();
  return unicInstancia; }
  
```

Client



1

Singleton.getInstancia().operacionSingleton()

```

if (instance == null) {
  // Nota: si estás creando una aplicación
  // que soporte el multihilo, debes
  // colocar un bloqueo de hilo aquí.
  instance = new Singleton()
}
return instance
  
```

Participantes:

1. Clase Singleton: define una operación estática *getInstancia()* para permitir acceder a la única instancia, que se conserva como un atributo de la clase (también estático). También define otros atributos que son los datos necesarios globalmente (en las aplicaciones suele ser uno solo).

El constructor del Singleton debe ocultarse del código cliente. La llamada a *obtenerInstancia* debe ser la única manera de obtener el objeto de Singleton.

Motivación: un manejador de ventanas, una cola de impresión, un sistema de ficheros, acceso BD.

Implementación:

1. **Constructor Privado:** Impide que otras clases creen instancias de Singleton directamente, asegurando que solo exista una instancia única.

2. **Referencia Única:** La clase mantiene una referencia estática (*unicaInstancia*) a su única instancia, inicializada solo cuando se llama a *getInstancia()* por primera vez.
3. **Método *getInstancia()*:** Si *unicaInstancia* es null, crea la instancia única; de lo contrario, devuelve la referencia existente, garantizando una única instancia accesible.
4. **Acceso a los Datos:** El cliente usa *getInstancia()* para acceder a métodos y datos del Singleton, como *mostrarDatoSingleton()* o *tomarDatoSingleton(dato)*.
5. **Registro de Singleton (opcional):** Permite asociar instancias únicas a subclases específicas, en caso de que se necesite.

Aplicabilidad: Útil para **recursos compartidos** como conexiones de base de datos, configuraciones de aplicaciones, etc.

Debe existir una **única instancia de una clase** y se necesita que **sea accesible por todas las demás del sistema**. Se define como clase pública, para brindar alcance global y que desde cualquier parte del código se obtenga visibilidad de la instancia Singleton con *Singleton.getInstancia()*.

Consecuencias:

- El acceso controlado a la instancia única, y al dato que guarda.
- Se evitan las variables globales → problema en las aplicaciones.
- La clase Singleton puede tener subclases, armando una jerarquía globalmente accesible.
- Este **patrón puede presentar problemas en accesos multihilo debido a la posibilidad de crear interbloqueos**, dependiendo de la implementación.

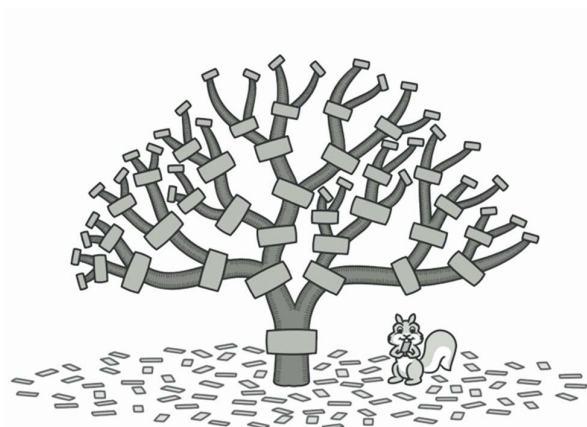
Principios de diseño OO se aplican en el patrón Singleton

Principio de responsabilidad única: al mantener una única instancia de la clase y dar acceso global, cumple contener “una sola razón para el cambio” y su responsabilidad principal es manejar la instancia única y el dato que guarda.

Principio de abierto/cerrado: el Singleton es una implementación cerrada, que encapsula en sí misma la lógica para crear y gestionar la instancia, sin embargo, está abierto a la extensión, ya que puede proporcionar métodos para accederla y permitir que los objetos actualicen el valor que almacena.

Principio Don't Repeat Yourself (DRY): el patrón Singleton sigue este principio al garantizar la instancia única, lo que permite que no se duplique el código y lógica relacionada al dato que almacena.

Principio Tell Don't Ask (TDA): proporciona un punto de acceso global para interactuar con él, fomentando la comunicación por mensajes en lugar de exponer sus detalles internos.



PATRÓN COMPOSITE.

Ámbito→OBJETO.

Propósito→ESTRUCTURAL.

Problema: Representar **estructuras jerárquicas** de objetos donde los **clientes necesitan tratar objetos individuales y composiciones** de objetos **de manera uniforme**.



El patrón Composite te permite ejecutar un comportamiento de forma recursiva sobre todos los componentes de un árbol de objetos.

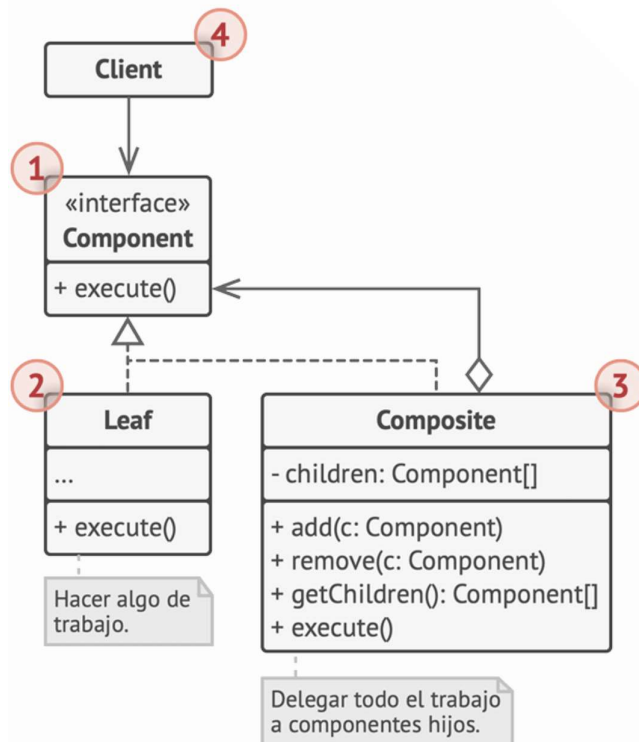
Propósito: Es un patrón de diseño estructural que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

Un **pedido** puede incluir varios **productos** empaquetados en **cajas**, que a su vez están empaquetados en cajas más grandes y así sucesivamente. La estructura se asemeja a un **árbol boca abajo**.

Solución: El patrón Composite sugiere que trabajes con Productos y Cajas a través de una **interfaz común** que declara un método para calcular el precio total.

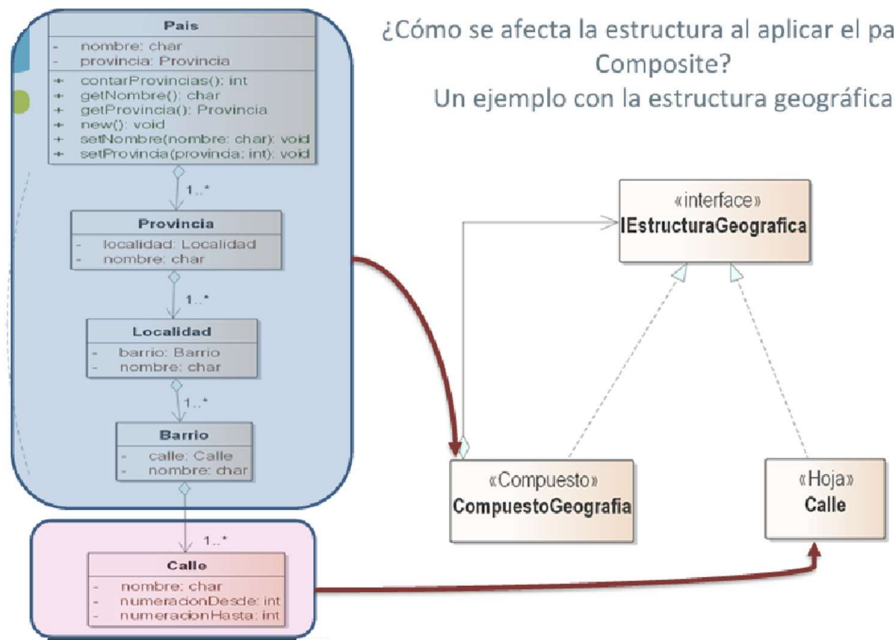
¿Cómo funcionaría este método? Para un **producto**, sencillamente **devuelve el precio del producto**. Para una **caja**, **recorre cada artículo que contiene la caja, pregunta su precio y devuelve un total** por la caja.

Participantes:



1. La **interfaz Componente** describe **operaciones** que son **comunes** a elementos simples y complejos del árbol.
2. La **Hoja es un elemento básico** de un árbol que no tiene subelementos. *Normalmente, los componentes de la hoja acaban realizando la mayoría del trabajo real, ya que son la unidad mínima del árbol.*
3. El **Contenedor (también llamado compuesto)** es un elemento que **tiene subelementos: hojas u otros contenedores**. Un contenedor no conoce las clases concretas de sus hijos. Funciona con todos los subelementos únicamente a través de la interfaz componente. Al recibir una solicitud, un **contenedor delega** el trabajo a sus subelementos, **procesa** los **resultados intermedios** y **devuelve** el resultado **final** al cliente.

4. El **Cliente funciona** con todos los elementos **a través de la interfaz componente**. Como resultado, el cliente puede funcionar de la misma manera tanto con elementos simples como complejos del árbol.



Aplicabilidad.

Útil para **estructuras que pueden representarse como árboles o jerarquías todo/parte** (por ejemplo, menús, sistemas de archivos). Se quiere que los **clientes ignoren la diferencia entre objetos compuestos y los objetos individuales** que los forman.

Consecuencias.

- ✓ Jerarquía con clases que modelan objetos primitivos y objetos compuestos, que permite **composición recursiva**.
- ✓ Clientes pueden **tratar objetos primitivos y compuestos de modo uniforme**.
- ✓ Es fácil añadir nuevos tipos de componentes.
- ✓ No se puede confiar al sistema de tipos que asegure que un objeto compuesto sólo contendrá objetos de ciertas clases, **necesidad de comprobaciones en tiempo de ejecución**.

Implementación.

1. **Estructura de árbol:** Asegúrate de que el modelo central de tu aplicación pueda representarse como una estructura de árbol, dividiéndolo en elementos simples y contenedores.
2. **Interfaz componente:** Declara una interfaz con métodos que sean útiles tanto para componentes simples como complejos.
3. **Clase hoja:** Crea una clase para representar elementos simples. Puede haber varias clases hoja diferentes.
4. **Clase contenedora:** Crea una clase para elementos complejos, con un campo matriz para almacenar referencias a subelementos (hojas y contenedores).
5. **Delegación de métodos:** Implementa los métodos de la interfaz en los contenedores, delegando la mayor parte del trabajo a los subelementos.
6. **Gestión de hijos:** Define métodos para añadir y eliminar elementos hijos dentro del contenedor.

Recuerda que estas operaciones pueden declararse en la interfaz componente, aunque esto podría violar el **Principio de segregación de la interfaz**. Sin embargo, permite tratar a todos los elementos de la misma manera al componer el árbol.

Relación con otros patrones. Puedes utilizar **Builder** al crear **árboles Composite** complejos porque puedes programar sus pasos de construcción para que funcionen de forma recursiva. Puedes utilizar **Iterators** para recorrer **árboles Composite**.

PATRÓN ADAPTER.

Ámbito→OBJETO y CLASE?

Propósito→ESTRUCTURAL.

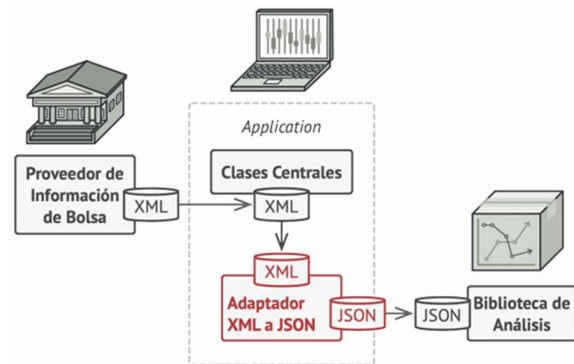
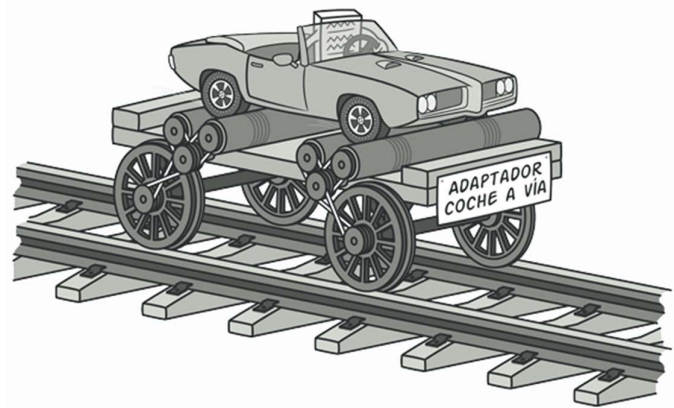
Problema: Necesitas **integrar dos interfaces incompatibles** sin modificarlas.

Propósito: Permite que las interfaces incompatibles trabajen juntas al **traducir una interfaz en otra**. Es un **patrón de diseño estructural** que permite la **colaboración entre objetos** con interfaces incompatibles.

Se dice que el **Adaptador “envuelve” uno de los objetos para ocultar la complejidad de la conversión**. El objeto “envuelto” no necesita conocer que se realiza una conversión.

Solución: Crear un objeto *Adaptador* que convierte la interfaz de un objeto, de forma que otro objeto pueda comprenderla. Los adaptadores **no solo convierten datos a varios formatos**, sino que también ayudan a objetos con distintas interfaces a colaborar, de esta forma:

1. El adaptador obtiene una interfaz compatible con uno de los objetos existentes.
2. Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en un formato y orden que ese segundo objeto espera.



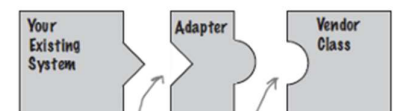
Motivación. Facilita la integración de bibliotecas de terceros o código heredado sin modificaciones.

Aplicabilidad. Se desea crear una clase existente y su **interfaz no concuerda con la necesaria**, o se desea crear una **clase reusable que debe cooperar con clases no relacionadas**, cuya relación no fue prevista.

Aplicabilidad

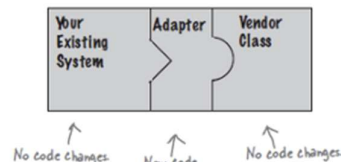


Their interface doesn't match the one you've written your code against. This isn't going to work



The adapter implements the interface your classes expect.

And talks to the vendor interface to service your requests.



Consecuencias. Depende del tipo de adaptador.

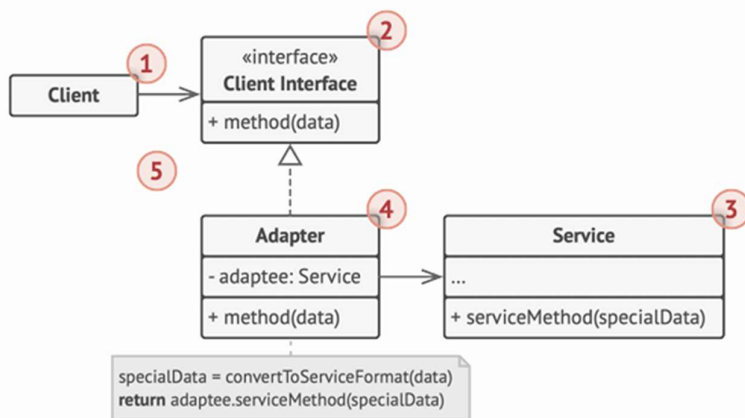
- ❖ **Un adaptador de objetos** permite adaptar una clase y sus subclases, pudiendo añadir funcionalidad a todos los adaptados, pero **dificulta redefinir el**

comportamiento de los adaptados. Usa el principio de composición: implementa la interfaz del objeto objetivo y envuelve al otro (adaptable).

- ❖ **Un adaptador de clases no funciona al intentar usarlo para clases y subclasses,** pero permite redefinir comportamiento heredado de la clase adaptada, sólo introduce un objeto sin necesidad de indirección adicional para obtener el adaptado. Usa la herencia, **el adaptador hereda las interfaces de ambos objetos por lo que solo es implementable en lenguajes que acepten herencia múltiple.**

- ✓ La funcionalidad del Adaptador depende de qué tan similares sean las interfaces de las clases Objetivo y Adaptable.
- ✓ La complejidad del código aumenta al introducir un conjunto de clases e interfaces nuevas, a veces puede resultar más simple cambiar la clase de servicio para que coincida con el código incompatible.

Participantes (Adapter de objetos).



1. La clase Cliente contiene la lógica de negocio existente del programa.

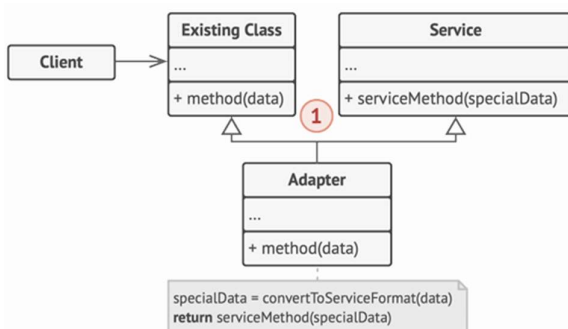
2. La Interfaz con el Cliente describe un protocolo que otras clases deben seguir para poder colaborar con el código cliente.

3. Servicio es alguna clase útil (normalmente de una tercera parte o heredada). El cliente no puede utilizar directamente esta clase porque tiene una interfaz incompatible.

4. La clase Adaptadora es capaz de trabajar tanto con la clase cliente como con la clase de servicio: implementa la interfaz con el cliente, mientras envuelve el objeto de la clase de servicio. La clase adaptadora recibe llamadas del cliente a través de la interfaz adaptadora y las traduce en llamadas al objeto envuelto de la clase de servicio, pero en un formato que pueda comprender.

5. El código cliente no se acopla a la clase adaptadora concreta siempre y cuando funcione con la clase adaptadora a través de la interfaz con el cliente. Gracias a esto, puedes introducir nuevos tipos de adaptadores en el programa sin descomponer el código cliente existente. Esto puede resultar útil cuando la interfaz de la clase de servicio se cambia o sustituye, ya que **puedes crear una nueva clase adaptadora sin cambiar el código cliente.**

Participantes (Clase adaptadora).



1. La Clase adaptadora no necesita envolver objetos porque hereda comportamientos tanto de la clase cliente como de la clase de servicio. La adaptación tiene lugar dentro de los métodos sobrescritos. La clase adaptadora resultante puede utilizarse en lugar de una clase cliente existente.

Sólo puede implementarse en lenguajes de programación que soporten la herencia múltiple (C++).

Implementación.

1. **Clases con interfaces incompatibles:** Asegúrate de tener al menos dos clases con interfaces incompatibles: una clase de servicio útil que no puedes cambiar y una o varias clases cliente que se beneficiarían de esa clase de servicio.
2. **Interfaz con el cliente:** Declara una interfaz que describa cómo las clases cliente se comunicarán con la clase de servicio.
3. **Clase adaptadora:** Crea una clase adaptadora que siga la interfaz con el cliente, inicialmente con métodos vacíos.
4. **Referencia al objeto de servicio:** Añade un campo en la clase adaptadora para almacenar una referencia al objeto de servicio, generalmente inicializado a través del constructor.
5. **Implementación de métodos:** Implementa los métodos de la interfaz con el cliente en la clase adaptadora, delegando la mayor parte del trabajo al objeto de servicio y gestionando la interfaz o la conversión de formato de datos.
6. **Uso de la clase adaptadora:** Las clases cliente deben utilizar la clase adaptadora a través de la interfaz con el cliente, permitiendo cambiar o extender las clases adaptadoras sin afectar al código cliente.

Principios de diseño OO se aplican en el patrón Adapter.

Segregación de interfaces: la interfaz intermedia tiene los comportamientos que le van a servir al cliente y los que tiene que implementar el adaptador.

Principio de responsabilidad única: se puede separar la interfaz o el código de conversión de datos de la lógica de negocio principal del programa.

Principio de abierto/cerrado: se pueden introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre que funcionen con los adaptadores a través de la interfaz con el cliente.

PATRÓN FACTORY METHOD.

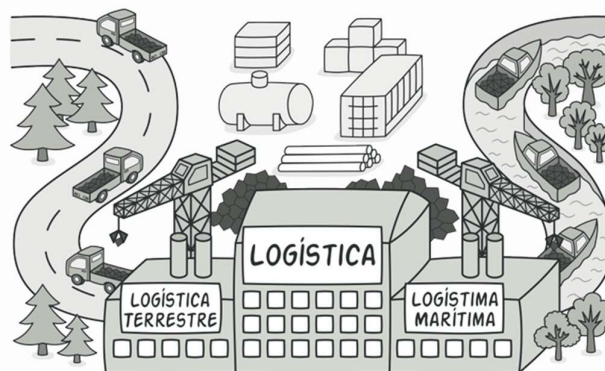
Ámbito→CLASE.

Propósito→CREACIÓN.

Problema: Decidir qué subclase instanciar en tiempo de ejecución puede llevar a un fuerte acoplamiento y complejidad.

Propósito: Define una interfaz para la creación de objetos delegando en las subclases la decisión de qué clase instanciar.

La intención del Factory Method es **tener una clase a la cual delegar la responsabilidad de la creación de los objetos**, para no decidir en tiempo de compilación que clase instanciará, si no que delegará esta responsabilidad al Factory confiando en que este le regresará la clase adecuada para trabajar.



Solucion: Define un método para crear objetos, pero permite a las subclases alterar el tipo de objetos que se crearán. **Proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.**

Motivacion. Permite que una clase **delegue la creación de objetos a sus subclases**, promoviendo la flexibilidad y escalabilidad.

- ❖ **Evita el acoplamiento estrecho** entre el creador y los productos concretos.
- ❖ **Principio de responsabilidad única.** Puede mover el código de creación del producto a un lugar en el programa, haciendo que el código sea más fácil de soportar.
- ❖ **Principio abierto / cerrado.** Puede introducir nuevos tipos de productos en el programa sin romper el código de cliente existente

Consecuencias.

- El código puede volverse más complicado ya que necesita introducir muchas subclases nuevas para implementar el patrón.
- El mejor de los casos es cuando se introduce el patrón en una jerarquía existente de clases de creadores.

Aplicabilidad.

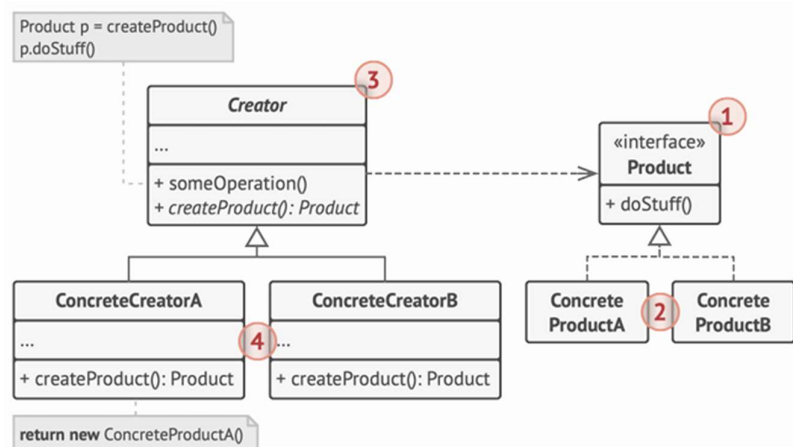
- ✓ **Para casos en los que no se sabe en tiempo de diseño que subtipo vamos a necesitar**
- ✓ Cuando una clase desea que sus subclases especifiquen los objetos que debe crear.
- ✓ **Cuando se desea crear tipos de objetos relacionados por una base común sin preocuparnos de los detalles de creación del objeto. La finalidad es que los mismos puedan variar sin afectar la forma en que el cliente interactúa con ellos.**

Implementación.

1. **Haga que todos los productos sigan la misma interfaz.** Esta interfaz debe declarar métodos que tengan sentido en cada producto.
2. **Agregue un método de fábrica vacío dentro de la clase creadora.** El tipo de retorno del método debe coincidir con la interfaz de producto común
3. **En el código del creador encuentre todas las referencias a los constructores de productos.** Uno por uno, *reemplácelos con llamadas al método de fábrica*, mientras extrae el código de creación del producto en el método de fábrica.
4. Si, después de todas las extracciones, el **método de fábrica base se ha quedado vacío, puede hacerlo *abstracto***. Si queda algo, puede convertirlo en un comportamiento predeterminado del método.

Participantes.

1.El Producto declara la interfaz, que es común a todos los objetos que puede producir la clase creadora y sus subclases.



2. Los Productos Concretos son distintas implementaciones de la interfaz de producto.

3. La clase Creadora declara el método fábrica que devuelve nuevos objetos de producto. Es importante que el tipo de retorno de este método coincida con la interfaz de producto.

4. Los Creadores Concretos sobrescriben el Factory Method base, de modo que devuelva un tipo diferente de producto.

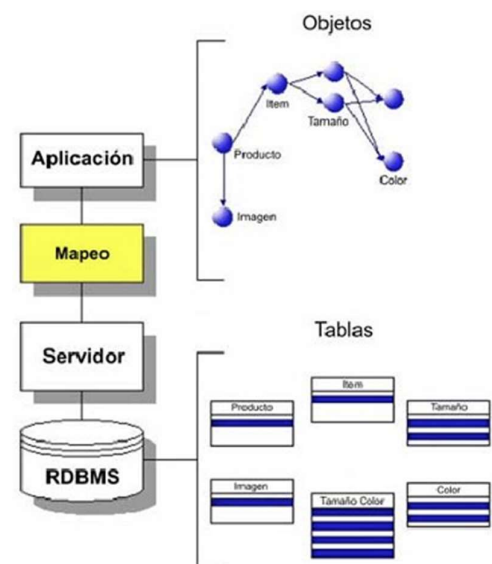
- **Declaración Abstracta:** Puedes declarar el método Factory Method como abstracto en la clase base. Esto obliga a todas las subclases a implementar sus propias versiones del método, asegurando que cada subclase pueda definir cómo se crea su propio tipo de producto.
- **Método Fábrica Base:** Alternativamente, el método fábrica en la clase base puede devolver un tipo de producto por defecto. Esto proporciona una implementación estándar que las subclases pueden optar por sobrescribir si necesitan un comportamiento específico.
- **Responsabilidad Principal:** A pesar de su nombre, la creación de productos no es la principal responsabilidad de la clase creadora. Normalmente, esta clase tiene alguna lógica de negocios central relacionada con los productos. El patrón Factory Method ayuda a desacoplar esta lógica de las clases concretas de producto.

Analogía: Imagina una gran empresa de desarrollo de software que tiene un departamento de formación de programadores. Aunque la empresa puede formar programadores, su función principal sigue siendo escribir código, no preparar programadores. De manera similar, una clase creadora puede tener la capacidad de crear productos, pero su responsabilidad principal es manejar la lógica de negocios.

DISEÑO DE PERSISTENCIA.

Persistencia en un sistema de información **se refiere a la capacidad de los datos para sobrevivir a la terminación del proceso que los creó**. Es decir, los datos persisten incluso después de que el programa que los generó haya dejado de ejecutarse, **permitiendo que sean recuperados y utilizados en ejecuciones futuras**. Se necesita **un mecanismo para almacenar y recuperar** objetos de ciertas clases, los mecanismos principales incluyen:

- **Bases de Datos Orientadas a Objetos (BDOO):** Permiten almacenar y recuperar objetos directamente, **sin transformaciones** adicionales, manteniendo la integridad del diseño orientado a objetos.
- **Bases de Datos Relacionales (BDR):** Predominan en el almacenamiento de datos, pero el modelo relacional **presenta incompatibilidades** con el modelo de objetos (herencia, asociaciones), conocido como el **problema de impedancia**. Este problema requiere una **capa de mapeo para transformar los objetos a estructuras relacionales y viceversa**, preservando el estado e identidad de los objetos.



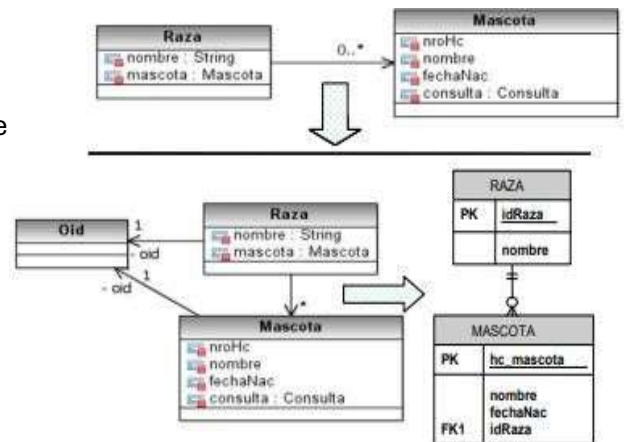
- **Otros Mecanismos:** Incluyen almacenamiento en archivos, XML, o bases de datos jerárquicas, que también requieren servicios especiales para traducir y manejar representaciones de objetos.

En el ámbito de la programación orientada a objetos (POO), uno de los desafíos significativos es el **problema de impedancia** entre el modelo de objetos y el modelo relacional de las bases de datos (RDBMS). Este problema surge porque los conceptos de POO, como herencia, encapsulamiento y asociaciones, no se mapean directamente a las estructuras de datos relacionales, lo que **complica la persistencia de los objetos**. Para abordar esto, es necesario implementar:

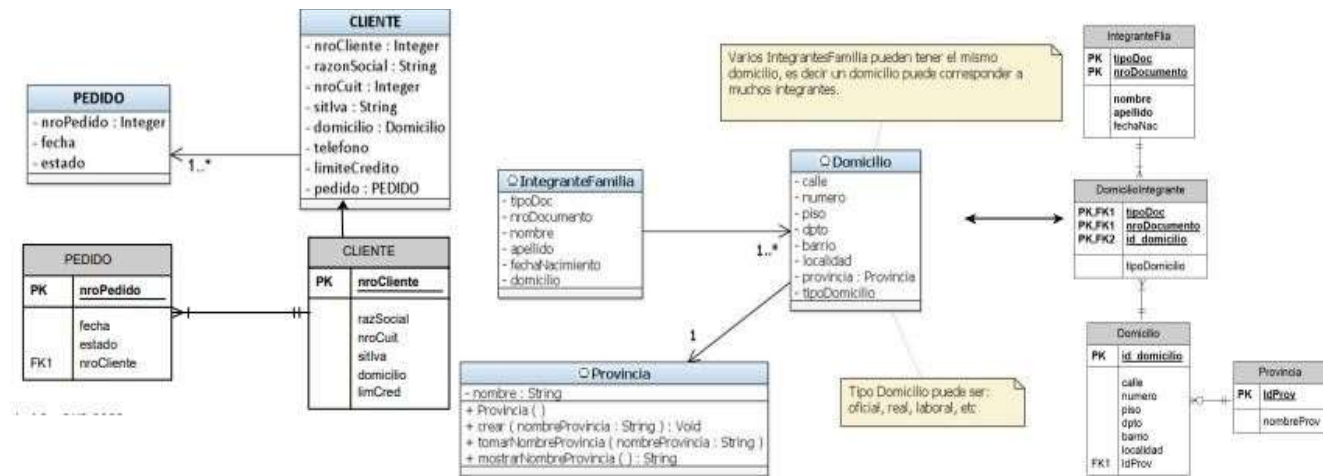
- **Materialización/Desmaterialización:** convertir registros de una base de datos en objetos y viceversa.
- **Mapeo Objeto-Relacional:** Transformar clases en tablas, atributos en columnas y gestionar la identidad de objetos mediante claves primarias.
- **Mapeo de Relaciones:** Traducir asociaciones y composiciones a tablas adicionales y manejar herencia mediante simulación, ya que las BDR no la soportan nativamente.
- **Violación del Encapsulamiento:** Atributos privados deben exponerse para ser almacenados en la BDR.

Mapeo de Clases a Bases de Datos Relacionales.

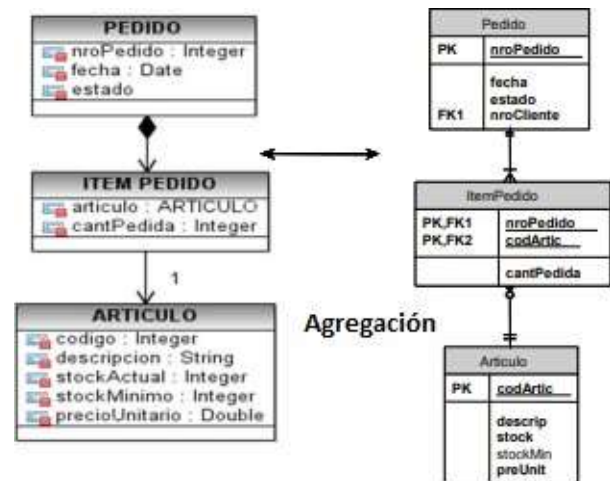
1. **Objetos en Tablas:** Cada clase persistente se convierte en una tabla. Los objetos de negocio son los principales candidatos para persistencia, ya que suelen sobrevivir a los casos de uso, a diferencia de las clases de fabricación pura. Las reglas son:
 - Si la clase contiene **solo atributos de datos primitivos**, se mapea directamente a una tabla con correspondencia uno a uno.
 - Si un atributo **contiene múltiples valores o referencias a otras clases**, se crean tablas adicionales para representar estas relaciones, conectadas por claves foráneas (FK).
 - Las **instancias de clases se almacenan como filas** en la tabla correspondiente, haciendo que los registros sean equivalentes a los objetos.
2. **Patrón Identificador de Objeto (OID):** Cada registro en una tabla tiene una clave primaria (PK) **única que representa el identificador del objeto**. Este patrón asegura la **unicidad de cada objeto**, permitiendo que el modelo orientado a objetos y el modelo relacional mantengan consistencia:
 - La clase OID contiene un atributo de tipo String que representa el nombre de la PK en la base de datos. Esto permite **encapsular el identificador en el modelo de objetos**.



3. Mapeo de Relaciones entre Objetos y Tablas:



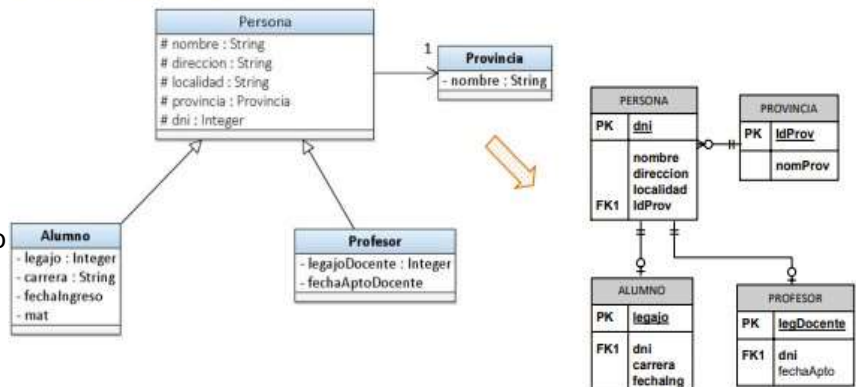
- **Asociaciones uno a muchos (1...*)**: La tabla con cardinalidad 1 contiene una FK que referencia la tabla con la cardinalidad de muchos, o bien se crea una tabla de asociación que contiene las PK de ambas tablas.
- **Asociaciones uno a uno (1...1)**: Se asigna una FK en una o ambas tablas para conectar las PK de cada objeto, o se crea una tabla de asociación específica.
- **Asociaciones muchos a muchos (M...N)**: Se utiliza una tabla de asociación que conecta las PK de cada clase relacionada, permitiendo la referencia mutua entre las tablas.
- **Relaciones de Agregación y Composición**: Estas relaciones se gestionan mediante tablas intermedias o referencias de FK que mantienen la relación de "todo-parte".



4. Mapeo de Herencia en un Modelo Relacional: Las bases de datos relacionales **no** soportan la herencia de forma nativa, por lo que se utilizan tres alternativas:

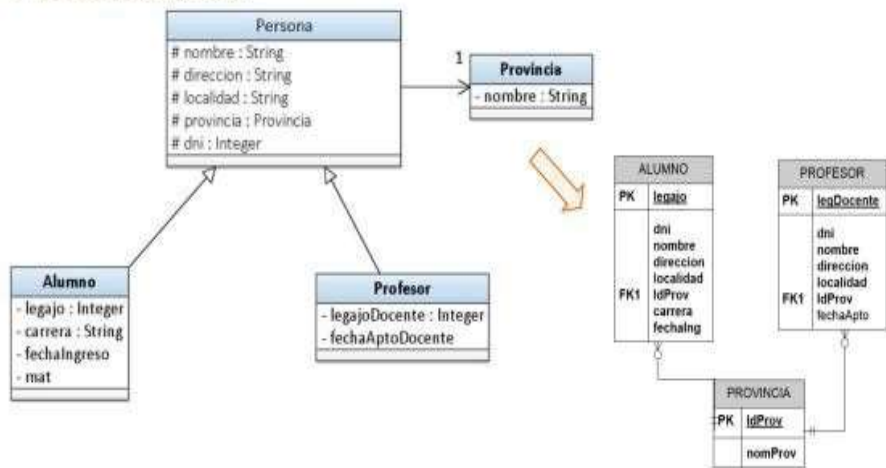
- **Simulación de Herencia**: Se crea una tabla para cada clase, donde la tabla hija tiene una FK que referencia la tabla padre, incluyendo solo los atributos adicionales de la clase hija. Esta opción reduce redundancia, pero requiere JOINS, afectando la performance.

• Alternativa 1:



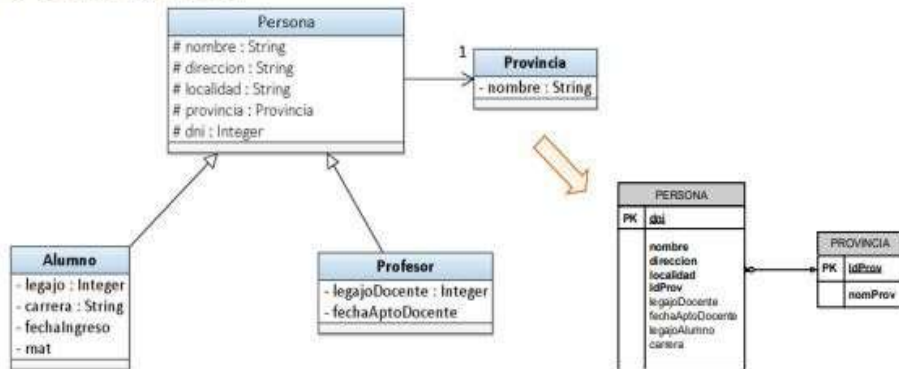
- **Eliminar la Tabla del Padre:** Solo se crean tablas para las clases hijas, duplicando atributos del padre. Esto mejora la velocidad de consulta evitando JOINS, pero aumenta el tamaño de la base de datos y el acoplamiento.

• Alternativa 2:



- **Eliminar Tablas de los Hijos:** Todos los atributos de las clases hijas se almacenan en la tabla del padre, usando campos nulos para atributos específicos de cada hijo. Esto reduce la cohesión y no es recomendable.

• Alternativa 3:



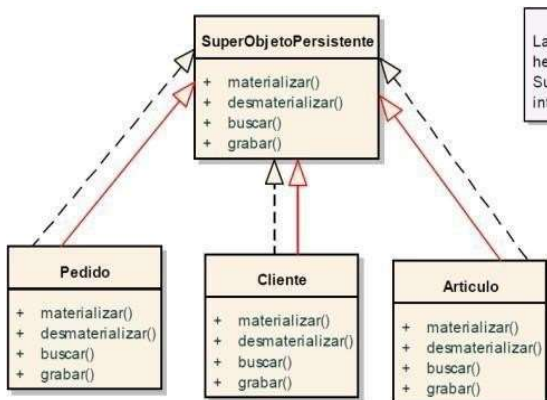
5. Transformación de Atributos y Relaciones:

- **Cada atributo de una clase se convierte en una columna de la tabla.** Las asociaciones con cardinalidad mayor que uno se gestionan con tablas adicionales que conectan las tablas originales, resolviendo la diferencia estructural entre los modelos de objetos y relacional.

DISEÑO DE PERSISTENCIA.

Para resolver la persistencia, se establece una correspondencia entre **clases y tablas**, y **atributos y campos** en el almacenamiento de datos, usando dos enfoques:

1. Super Objeto Persistente:



La realización reemplaza a la herencia (en rojo). Conviertiendo al *SuperObjetoPersistente* en una interfaz.

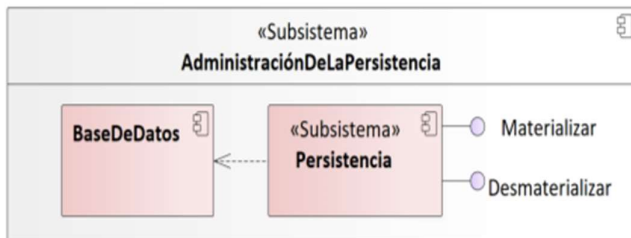
Consiste en una clase abstracta, **SuperObjetoPersistente**, que define métodos para materialización y desmaterialización. Todas las **clases que requieren persistencia heredan de esta clase**, permitiendo la **correspondencia directa** sin una capa intermedia.

- **Ventajas:** Solución sencilla y ordenada, **no incrementa el número de**

clases.

- **Desventajas:** Alto acoplamiento entre la capa de lógica de negocio y la de datos; en lenguajes con herencia simple consume la única posibilidad de herencia.
- **Aplicación:** Se encuentra en la capa de lógica de negocio y permite que las clases persistentes se guarden y recuperen directamente desde la base de datos.

2. Servicio de Persistencia para BDR (Framework de Persistencia):



Se utiliza un **framework de persistencia que proporciona funciones reutilizables y extensibles** para almacenar y recuperar objetos, gestionar transacciones y separar la lógica de negocio de la capa de datos.

- **Ventajas:** Fácil de extender, bajo acoplamiento y alta cohesión.
- **Desventajas:** Complejo de implementar y más lento que un super objeto persistente debido a la capa adicional.
- **Elementos Clave:**
 - **Materialización/Desmaterialización:** Transforma registros a objetos y viceversa.
 - **Caché:** Almacena objetos materializados para mejorar el rendimiento.
 - **Materialización Perezosa:** Los objetos solo se recuperan cuando es necesario.
 - **Operaciones de Transacción:** Confirma o deshace transacciones (commit/rollback), asegurando consistencia en el estado del objeto.
 - **Correspondencia Indirecta:** Agrega una capa entre la lógica de negocio y la base de datos para reducir el acoplamiento, aplicando el **Principio de Hollywood (DIP)**. Donde una clase de control dentro del esquema invoca la persistencia, y cuando esto pasa el esquema de persistencia toma el control, donde busca el objeto que hay que hacer persistente y lo hace persistente sobre la base de datos, **no nos llames, nosotros te llamamos, de esta forma mantengo la cohesión** de las clases de dominio porque no las tengo que tocar, la persistencia se encarga de eso. Tiene bajo acoplamiento.
Hay una **dependencia entre la persistencia y la BD**, si la cambio debo modificar el esquema de persistencia.

Patrones de Persistencia

1. Patrón Identificador de Objetos (OID):

- **Descripción:** Asigna un **identificador único a cada objeto y registro** en la base de datos, permitiendo que se relacionen sin duplicados.
- **Función:** Asegura la **consistencia y unicidad** entre objetos y registros al evitar que se materialicen objetos duplicados.

2. Patrón Fachada (Facade):

- **Descripción:** Proporciona una **interfaz uniforme para acceder a los servicios de persistencia**, delegando el trabajo en otros objetos del subsistema.
- **Función:** Simplifica el **acceso al sistema de persistencia mediante una interfaz centralizada**, útil para recuperar objetos a partir de su OID.

3. Patrón Conversor (Mapper) o Intermediario (Broker):

- **Descripción:** Crea una **clase que se encarga de materializar y desmaterializar objetos**, creando un intermediario para cada clase de dominio.
- **Función:** Facilita la **correspondencia entre objetos y tablas**, usando una capa intermedia para gestionar la persistencia.

4. Patrón Template Method para Materialización:

- **Descripción:** Utiliza el método plantilla para **encapsular el comportamiento común de la materialización** en una clase base, permitiendo variaciones en la forma de crear el objeto desde el almacenamiento.
- **Función:** Estandariza el proceso de materialización con un punto de variación para ajustar cómo se crean objetos específicos.

5. Patrón Gestión de Caché:

- **Descripción:** **Almacena** objetos materializados **en una caché local**, mejorando el rendimiento y soportando la gestión de transacciones, **utilizando el OID como clave**.
- **Función:** Reduce las materializaciones innecesarias al buscar primero en la caché, optimizando el acceso a datos ya materializados.

6. Patrón State para Estados Transaccionales:

- **Descripción:** Utiliza el **estado del objeto para decidir cuándo realizar una operación sobre la base de datos**, controlando las inserciones, eliminaciones y modificaciones en función de su estado de transacción.
- **Función:** Gestiona el ciclo de vida de las transacciones, asegurando que las operaciones sobre los objetos persistentes se realicen de manera coherente y ordenada.

7. Patrón Command para Transacciones:

- **Descripción:** **Representa cada operación de una transacción como un objeto** con un método ejecutar(), permitiendo que se ordenen en una cola y se apliquen o deshagan (rollback) según sea necesario.
- **Función:** Permite **gestionar operaciones atómicas en el sistema de persistencia**, asegurando que las tareas se realicen en su totalidad o no se realicen, manteniendo la consistencia de la base de datos.

8. **Proxy Virtual para Materialización Perezosa:**

- **Descripción:** Un proxy inteligente que **materializa un objeto solo cuando se accede a él por primera vez**, utilizando su OID para identificar y recuperar el objeto real.
- **Función:** **Optimiza el uso de recursos** al retrasar la materialización de los objetos hasta que realmente se necesiten, mejorando la eficiencia.